



Hoy en día Java es un lenguaje
mucho más moderno y atractivo de
lo que era hace un tiempo atrás

Java

EL CAMBIO ES LA ÚNICA CONSTANTE

**JORDY ANDRÉS
RODRÍGUEZ ARANGO**

**4 SOFTWARE
DEVELOPERS**

Java, el cambio es la única constante Segunda Edición - 2025

© 2025 **4SoftwareDevelopers** Todos los derechos reservados.

Autor: Jordy Andrés Rodríguez Arango **Edición y Revisión:** Jordy Andrés Rodríguez Arango

Diseño de Portada: Nano Banana Pro a partir de imagen original.

Licencia de Distribución: Esta obra se distribuye bajo una licencia **Creative Commons**
Atribución-NoComercial-SinDerivadas 4.0 Internacional (CC BY-NC-ND 4.0).

Usted es libre de:

- **Compartir:** copiar y redistribuir el material en cualquier medio o formato.

Bajo las siguientes condiciones:

- **Atribución:** Debe dar el crédito apropiado al autor, brindar un enlace a la licencia e indicar si se realizaron cambios.
- **No Comercial:** No puede hacer uso del material con fines comerciales (nadie puede vender este libro).
- **Sin Derivadas:** Si remezcla, transforma o crea a partir del material, no puede distribuir el material modificado.

Nota del Autor: A partir de Diciembre de 2025, este libro es **100% gratuito**. Si has pagado por esta copia, has sido víctima de una estafa. Puedes descargar la versión original y actualizada de forma gratuita en los canales oficiales de **4SoftwareDevelopers**.

Descargo de Responsabilidad: La información contenida en este libro se proporciona "tal cual", sin garantías de ningún tipo. Aunque se han tomado todas las precauciones en la preparación de este contenido, el autor y el editor no asumen ninguna responsabilidad por errores u omisiones, ni por daños resultantes del uso de la información contenida aquí. El código se proporciona con fines educativos.

Contacto: Para comentarios, dudas o correcciones: ebooks@4softwaredevelopers.com

Síguenos: YouTube: [4SoftwareDevelopers](https://www.youtube.com/4SoftwareDevelopers) Twitter/X: [@jordy_4sd](https://twitter.com/jordy_4sd)

Acerca del autor

Prefacio a la Segunda Edición

Introducción

01. Un Ecosistema sin Fronteras

02. El Renacimiento: De Dinosaurio a Fórmula 1

03. El "Caos" de las Versiones: Entendiendo el Nuevo Ciclo

04. Java 8: El punto de inflexión

1. Expresiones Lambda

2. Stream API

3. Optional: El fin (o el inicio del fin) de NullPointerException

4. Interfaces: Métodos Default y Static

5. Interfaces Funcionales

6. La Nueva API de Fecha y Hora (java.time)

7. Programación Asíncrona con CompletableFuture

Conclusión: El Nuevo Estándar Mínimo

05. Java 9: La Arquitectura del Futuro

1. El Sistema de Módulos (Project Jigsaw)

2. Colecciones Inmutables (Factory Methods)

3. JShell: Java Interactivo (REPL)

4. Mejoras en Stream API

5. Métodos Privados en Interfaces

Conclusión

06. Java 10: La Revolución de la Inferencia de Tipos

1. var: Inferencia de Tipos en Variables Locales

2. Colecciones Realmente Inmutables (copyOf)

3. Optional.orElseThrow() sin argumentos

4. Mejoras de Rendimiento (G1GC)

Conclusión

07. Java 11: El Primer LTS Moderno

1. El Nuevo Cliente HTTP (Standard HTTP Client)

2. Adiós a la "Ceremonia": Ejecución de Archivos Fuente

3. Métodos Vitamina para la clase String

4. var en Parámetros de Lambda

Conclusión

08. Java 12: Switch Inteligente y Colectores Potentes

1. El Nuevo Switch (Switch Expressions - Preview)

2. Collectors.teeing() (joya oculta)

3. Nuevos Métodos para String

4. Formato de Números Compactos

Conclusión

09. Java 13: Bloques de Texto y Cimientos del Futuro

1. Bloques de Texto (Text Blocks - Preview)

2. Refinando el Switch: La llegada de yield

3. Reimplementación de la API de Sockets (JEP 353)
 4. ZGC: Devolviendo memoria al sistema
 - Conclusión
10. Java 14: El Fin del "Boilerplate"
1. Switch Expressions (Aquí se hizo oficial)
 2. Records (Preview): Datos, no burocracia
 3. Pattern Matching para instanceof (Preview)
 4. NullPointerExceptions (NPE) Útiles
 - Conclusión
- 11 Java 15: Control de Herencia y Rendimiento Puro
1. Sealed Classes (Preview): El fin de la herencia descontrolada
 2. Bloques de Texto (¡Ya es oficial!)
 3. ZGC: Listo para Producción
 4. Hidden Classes (Clases Ocultas)
 - Conclusión
12. Java 16: Estabilizaciones interesantes
1. Records y Pattern Matching (¡Ya son oficiales!)
 2. jpackage: Instaladores Nativos
 3. Vector API (Incubator)
 4. Invocación de Métodos Default vía Reflexión
 5. Momentos del día (B pattern)
 - Conclusión
13. Java 17: El Nuevo Estándar de la Industria
1. Sealed Classes (Oficialmente Estándar)
 2. Encapsulamiento Fuerte de los Internos del JDK (JEP 403)
 3. Generadores de Números Pseudo-Aleatorios (JEP 356)
 4. HexFormat (Utilidad para Hexadecimales)
 5. Pattern Matching para switch (Preview)
 6. Limpieza de la Casa: Adiós Applets
 - Conclusión
14. Java 18: Adiós a los Problemas de Caracteres
1. UTF-8 por Defecto (JEP 400)
 2. Simple Web Server (JEP 408)
 3. Javadoc @snippet (JEP 413)
 - Conclusión
15. Java 19: El Prólogo de la Revolución
1. Virtual Threads (Preview): El fin de la espera
 2. Record Patterns (Preview): Deconstrucción de Datos
 3. Structured Concurrency (Incubator)
 4. Pattern Matching para Switch (Third Preview)
 - Conclusión
16. Java 20: La "calma" antes de la tormenta
1. Scoped Values (Incubator): El reemplazo de ThreadLocal

- 2. Record Patterns: Inferencia de Genéricos (2nd Preview)
- 3. Un consejo práctico: Deja de usar java.net.URL
- 4. Pattern Matching for Switch (4th Preview)
- Conclusión
- 17. Java 21: La Era de Oro (LTS)
 - 1. Virtual Threads (Oficial): Escalabilidad Infinita
 - 2. Sequenced Collections (Colecciones Secuenciadas)
 - 3. Pattern Matching Finalizado (Switch y Records)
 - 4. Simplificación del "Main" (Preview)
 - Conclusión
- 18. Java 22: Código Limpio y Sin Fronteras
 - 1. Variables y Patrones sin Nombre (␣) (JEP 456)
 - 2. Foreign Function & Memory API (FFM) - ¡Oficializada! (JEP 454)
 - 3. Statements before super(...) (Preview) (JEP 447)
 - 4. Stream Gatherers (Preview) (JEP 461)
 - Conclusión
- 19. Java 23: Documentación Humana y Simplicidad
 - 1. Javadoc en Markdown (JEP 467)
 - 2. Importación de Módulos (Preview) (JEP 476)
 - 3. Tipos Primitivos en Patrones (Preview) (JEP 455)
 - 4. Simplificación para Principiantes: IO (Third Preview)
 - Conclusión
- 20. Java 24: La Pieza Faltante de los Streams
 - 1. Stream Gatherers (Oficial)
 - 2. Class-File API (Preview Avanzada)
 - 3. Reducción de "Pinning" en Virtual Threads
 - 4. String Templates (El retorno)
 - Conclusión
- 21. Java 25: La Cuspide del Rendimiento (LTS)
 - 1. Value Classes (Clases de Valor) - Early Access/Standard
 - 2. Compact Object Headers (Project Lilliput)
 - 3. Pattern Matching en Bucles for
 - 4. Computed Constants (Project Leyden - Preview)
 - Conclusión del Viaje
- 22. GraalVM y AOT: La Revolución Nativa
 - 1. ¿Qué es AOT y GraalVM?
 - 2. El cambio de mentalidad: "Mundo Cerrado"
 - 3. Frameworks "Cloud Native"
 - 4. ¿Cuándo usar AOT y cuándo JIT?
 - Conclusión
- 23. Herramientas para Sobrevivir al Cambio
 - 1. SDKMAN!: Domina el Multiverso de Versiones
 - 2. OpenRewrite: Migración Automatizada

3. JBang: Scripts en Java sin dolor

Epílogo. Tu Hoja de Ruta Personalizada

Ruta 1: La Misión de Rescate (Migración)

Ruta 2: El Arquitecto de Software (Diseño)

Ruta 3: Rendimiento Cloud Native (Escalabilidad)

Palabras Finales

Acerca del autor



Soy **Jordy Rodríguez**, arquitecto de software y desarrollador apasionado, Fundador de *4SoftwareDevelopers*.

Llevo más de 11 años escribiendo código, rompiendo producción y volviéndola a arreglar (mejor que antes). He trabajado con todo tipo de stacks, desde Java y Spring hasta React Native y Node.js, lo que considero me ha dado ayudado a madurar mi visión, me gusta compartir lo que me ha funcionado y evito hablar de lo que no sé.

Mi misión con este libro es desafiar tus prejuicios sobre este lenguaje. Si eres de los que “huye” de Java por considerarlo verboso o anticuado, este libro te hará dudar. **No te prometo amor eterno, pero te aseguro que después de leerlo, Java volverá a ser una opción seria en tu stack tecnológico.**

Por otro lado, si eres de los que ama el lenguaje o simplemente trabajo con él, este libro se convertirá en tu herramienta predilecta para actualizar tus conocimientos y conocer los secretos “PRO” del lenguaje.

Prefacio a la Segunda Edición

La aceleración del cambio

Cuando publiqué la primera edición de este libro en 2019, mi intención era demostrar que Java había dejado de ser el lenguaje lento y verboso que muchos recordaban. Sin embargo, lo que ha ocurrido en el ecosistema desde entonces no ha sido una simple evolución; ha sido una aceleración total.

Desde la consolidación de **Java 21** como la versión LTS (Long Term Support) más robusta de la década, hasta las promesas de optimización extrema en **Java 24**, el lenguaje ha entrado en una nueva era. Ya no estamos hablando solo de sintaxis limpia; estamos hablando de un rendimiento capaz de competir en la era de la nube nativa, la Inteligencia Artificial y la concurrencia masiva.

Esta segunda edición nace de esa necesidad de actualización. Aquí no solo encontrarás corregidos y ampliados los conceptos fundamentales, sino que nos adentraremos en las tecnologías que están redefiniendo el backend moderno: desde la revolución de los **Virtual Threads** que cambian las reglas del juego en la escalabilidad, hasta la **API de Memoria y Funciones Foráneas (FFM)** y el **AOT (Ahead-of-Time)** que preparan a Java para un arranque instantáneo.

Escribí y actualicé este libro pensando especialmente en quienes creen que "odian" Java o que lo consideran una tecnología del pasado. Mi objetivo no es convertirme en un fanático ciego, ni prometer amor eterno por el lenguaje. Mi meta es que, al cerrar estas páginas, descubras que esa herramienta que quizás descartaste hace años, hoy se ha convertido en una pieza de ingeniería tan potente y moderna que, inevitablemente, querrás considerarla de nuevo en tu stack tecnológico.

Finalmente, gracias a la meta que conseguimos como comunidad (15 miembros de pago en [Youtube](#)) este eBook desde Diciembre de 2025 será **100% gratuito**. El conocimiento técnico de calidad no debería tener barreras.

Esta es mi contribución a la comunidad de **4SoftwareDevelopers** y a todos los programadores de habla hispana que, como yo, creen que en esta industria el cambio es la única constante.

Bienvenido a la versión más potente de Java... hasta la fecha.

Introducción

Java es el lenguaje de programación que mueve al mundo corporativo, pero mantener esa posición requiere algo más que inercia: requiere adaptación.

Muchos desarrolladores se quedaron "congelados" en la mentalidad de Java 8 (o incluso versiones anteriores). Siguen escribiendo código innecesariamente complejo, luchando contra `NullPointerException` que hoy son fáciles de evitar, y gestionando hilos del sistema operativo cuando la JVM ya ofrece soluciones mucho más sofisticadas.

La realidad que he visto de primera mano es que la brecha entre "saber Java" y "conocer el Java Moderno" es cada vez más grande. Mientras el lenguaje avanza hacia ciclos de actualización semestrales, muchos profesionales siguen programando como si estuviéramos en 2014.

Este material existe para cerrar esa brecha. No vamos a repasar la sintaxis básica que ya conoces. Vamos a explorar la transformación radical del lenguaje: desde la inferencia de tipos con `var`, pasando por la programación reactiva y funcional, hasta las optimizaciones de memoria de las últimas versiones LTS.

Java ha cambiado. La pregunta es: ¿has cambiado tú con él?

01. Un Ecosistema sin Fronteras

Probablemente tienes este libro en tus manos por una de tres razones.

Quizás eres un desarrollador con **curiosidad**, buscando una tecnología robusta y quieres saber si Java sigue siendo una apuesta segura para tu carrera. Tal vez eres un veterano que lleva años trabajando con el lenguaje, pero sientes que te has quedado **estancado** en versiones antiguas mientras el mundo avanza. O, seamos honestos, puede que seas un **escéptico** que ha escuchado que Java es "lento y verboso", y estás aquí para ver si eso ha cambiado o para confirmar tus sospechas.

Independientemente de cuál sea tu punto de partida, la realidad es la misma: **el Java que crees seguro que ha cambiado radicalmente.**

Ese lenguaje rígido y puramente corporativo del pasado ha dado paso a una plataforma moderna, ágil y sorprendentemente versátil. Hoy, Java no se trata de mantener sistemas legados; se trata de construir el futuro.

¿Qué puedes construir realmente hoy en día?

Existe la creencia de que Java está limitado a aplicaciones bancarias aburridas o sistemas de escritorio antiguos. Nada más lejos de la realidad. Si decides adentrarte en este ecosistema, descubrirás que las posibilidades son prácticamente infinitas:

- **La Web Moderna:** Es el motor detrás de muchas arquitecturas de microservicios escalables y de alto rendimiento que soportan el tráfico de los gigantes de internet.
- **Herramientas de Línea de Comandos (CLI):** Olvida las interfaces gráficas pesadas; Java es excelente para crear herramientas de automatización y scripts potentes que corren en cualquier sistema operativo.
- **Internet de las Cosas (IoT):** Desde dispositivos embebidos hasta sistemas de control industrial, la portabilidad de Java lo mantiene como un líder en el mundo del hardware conectado.

Una nota de honestidad técnica

No vengo a venderte humo. Java es poderoso, pero no es la única herramienta en la caja. Si tu objetivo principal es el entrenamiento puro de modelos de **Machine Learning** o IA, lenguajes como Python tienen un ecosistema de librerías más maduro para esa tarea específica. Sin embargo, cuando necesitas llevar esos modelos a producción, hacerlos escalables y seguros, Java podría ser un protagonista indiscutible

El objetivo de este libro

Este capítulo es una invitación a dejar atrás los prejuicios y el miedo. Tanto si vienes por curiosidad, por necesidad de actualización o por escepticismo, lo que encontrarás en las siguientes páginas es un lenguaje que ha sabido reinventarse.

Vamos a descubrir juntos por qué Java sigue siendo, y seguirá siendo, una de las habilidades más valiosas que puedes tener como desarrollador de software.

02. El Renacimiento: De Dinosaurio a Fórmula 1

Si le preguntas a un desarrollador que abandonó Java hace años por qué lo hizo, probablemente te dirá dos cosas: *"Es demasiado verboso (mucho código para hacer poco)"* y la clásica *"Evoluciona muy lento"*.

Y tendría razón... si estuviéramos en 2010.

Pero afirmar que Java es un lenguaje anticuado hoy en día es ignorar una de las transformaciones más agresivas en la historia del software. El lenguaje que antes tardaba 3 o 4 años en lanzar una actualización, hoy se renueva cada **seis meses**. Esto no es solo un cambio de calendario; es un cambio de filosofía que ha permitido incorporar características modernas a una velocidad vertiginosa.

El mito de la verbosidad

Una de las mayores críticas históricas a Java ha sido el "Boilerplate": tener que escribir getters, setters, constructores, `equals()`, `hashCode()` y `toString()` para una clase simple de datos.

Veamos cómo ha cambiado la historia.

El Pasado (Lo que crea el estigma): Para crear una clase inmutable por ejemplo un DTO que almacene datos de una transacción, antes necesitabas más de 50 líneas de código repetitivo y propenso a errores humanos. Era tedioso y aburrido.

El Presente (Java Moderno): Gracias a la llegada de los **Records** (lanzados en Java 16), todo ese ruido desaparece. Lo que antes ocupaba una pantalla entera, hoy se define en una sola línea concisa, segura y legible:


```
public record Transaccion(String id, double monto,  
    LocalDateTime fecha) {}
```

¡Eso es todo! En esa sola línea, Java te regala automáticamente el constructor, los métodos de acceso, la implementación de igualdad y la representación en texto.

Más allá de la estética

No se trata solo de escribir menos; se trata de potencia.

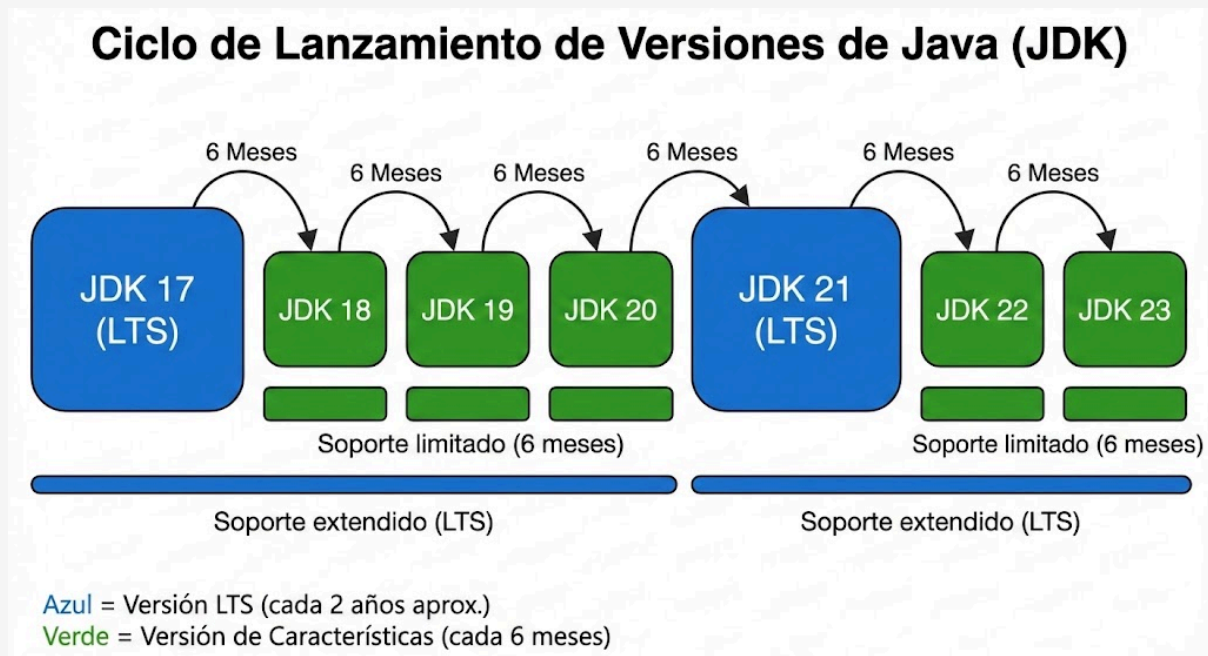
- ¿Te molestaba declarar tipos obvios? Con la inferencia de tipos (**var**), el compilador trabaja para ti.
- ¿Los hilos del sistema operativo colapsaron tu servidor? Los **Virtual Threads** de Java 21 permiten manejar millones de conexiones concurrentes con el mismo hardware, algo que en el pasado era impensable sin arquitecturas reactivas complejas.

Conclusión

Decir que Java es anticuado hoy es como decir que un coche eléctrico es lento porque los primeros modelos lo eran. El equipo de arquitectura de Java ha logrado lo imposible: modernizar la sintaxis y el rendimiento del lenguaje sin romper la compatibilidad con el código que mueve el mundo.

Java no se ha hecho viejo; al contrario ha mejorado con el tiempo y ahora sorprendentemente es mucho más rápido.

03. El "Caos" de las Versiones: Entendiendo el Nuevo Ciclo



Si vienes del mundo de Java 8 o anterior, es probable que te sientas abrumado. Antes, estábamos acostumbrados a esperar tres o cuatro años para una nueva versión del JDK. Era un evento masivo que cambiaba todo. De repente, parpadeaste y pasamos de Java 8 a Java 25.

¿Qué pasó? ¿Java se volvió loco?

No. Java maduró.

A partir de Java 9, Oracle y la comunidad cambiaron la estrategia radicalmente. Se dieron cuenta de que en el mundo moderno del desarrollo de software, esperar años para recibir nuevas características era inaceptable. Así nació el nuevo ciclo de lanzamientos semestrales (cada 6 meses).

Cómo funciona el tren de versiones

Para no perderte, solo necesitas entender dos conceptos clave: **LTS** y **Feature Releases**.

1. Las versiones LTS (Long Term Support) Estas son las versiones diseñadas para la estabilidad empresarial a largo plazo. Son las "rocas" sobre las que construyes sistemas que deben durar años en producción sin cambios drásticos.

- Se lanzan cada 2 años.
- Tienen soporte extendido y actualizaciones de seguridad por muchos años.
- **Ejemplos clave:** Java 8, Java 11, Java 17, Java 21 y la actual **Java 25**.

2. Las Feature Releases (Versiones de Innovación) Son todas las versiones intermedias entre cada release (como Java 22, 23, 24).

- Se lanzan cada 6 meses puntualmente.
- Su objetivo es introducir nuevas características rápidamente para que los desarrolladores las prueben.
- Tienen soporte corto (solo hasta que sale la siguiente versión).

¿Entonces, cuál debo usar?

Esta es la regla de oro moderna:

- **Para Producción:** Usa la última versión **LTS** disponible (actualmente **Java 25**). Es estable, segura y tiene soporte garantizado.
- **Para Desarrollo y Aprendizaje:** Mantente siempre en la última versión disponible. Al estar hoy en una LTS reciente, estás en el mejor de los mundos: estabilidad y las últimas novedades al mismo tiempo.

El mito del "Pago por Java"

Es importante aclarar algo que causó mucha confusión en el pasado. **Java sigue siendo gratis.** Aunque Oracle cambió sus licencias

comerciales hace años, el ecosistema se ha estandarizado alrededor del **OpenJDK**. Hoy en día, puedes descargar versiones gratuitas y de alto rendimiento de proveedores como Eclipse (Adoptium), Amazon (Corretto), Microsoft o Azul, todas basadas en el mismo código fuente del OpenJDK.

Conclusión

No hay caos, hay velocidad. El lenguaje ya no se estanca; evoluciona constantemente para que tú tengas mejores herramientas cada seis meses.

04. Java 8: El punto de inflexión

Si tuviéramos que dividir la historia de Java en dos eras, la línea divisoria sería, sin duda, la versión 8. Antes de ella, Java era puramente imperativo y orientado a objetos clásico. Después de ella, Java introdujo la programación funcional, los Streams y una concisión que no habíamos visto antes.

Aunque hoy estamos en Java 25, **todo lo que definimos como “Java Moderno” nació aquí**. Por eso, es fundamental entender estos cimientos, ya que muchas de las características más avanzadas de hoy (como los Records o los Virtual Threads) se construyeron sobre la filosofía que introdujo esta versión.

Repasemos los cambios que redefinieron el lenguaje:

1. Expresiones Lambda

Probablemente el cambio más icónico. Las lambdas nos permiten pasar funcionalidad como si fuera un argumento de un método (tratando el código como datos). Esto redujo drásticamente la necesidad de las clases anónimas verbosas.

El "Viejo" Java (Clase Anónima):

```
Runnable r1 = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Hola desde Runnable clásico");  
    }  
};
```

Java Moderno (Lambda):

```
Runnable r1 = () -> System.out.println("Hola desde Lambda");
```

La sintaxis es simple: `(argumentos) -> (cuerpo)`. Si el cuerpo es de una sola línea, te ahorras incluso las llaves `{}`.

2. Stream API

Esta es la joya de la corona de Java 8. El paquete `java.util.stream` cambió para siempre la forma en que trabajamos con colecciones. En lugar de iterar manualmente con bucles `for` o `while` y variables temporales, ahora declaramos *qué* queremos hacer con los datos.

Imagina que tienes una lista de tecnologías y quieres filtrar las que no sean "4sd".

Estilo Imperativo (Antes):

```
List<String> lines = Arrays.asList("spring", "node", "4sd");
List<String> result = new ArrayList<>();

for (String line : lines) {
    if (!"4sd".equals(line)) {
        result.add(line);
    }
}
```

Estilo Declarativo (Streams):

```
List<String> lines = Arrays.asList("spring", "node", "4sd");
List<String> result = lines.stream()
    .filter(line -> !"4sd".equals(line))
    .collect(Collectors.toList());
```

```
result.forEach(System.out::println);
```

Con Streams, el código se lee como una frase: *"Convierte a stream, filtra esto, y recolecta el resultado"*. Es más limpio, legible y menos propenso a errores lógicos.

Algo importante resaltar aquí es que ambos estilos siguen estando disponibles, no es obligación usar todo como lambdas o para todo usar Stream API, úsalo según tu contexto lo necesite.

3. Optional: El fin (o el inicio del fin) de NullPointerException

El manejo de nulos ha sido históricamente el mayor dolor de cabeza en Java. Java 8 introdujo `Optional<T>`, un contenedor que puede o no contener un valor no nulo. Su objetivo es evitar que `NullPointerException` te explote en la cara y forzarte a pensar: *"¿Qué pasa si esto viene vacío?"*.

Creación y Uso:

```
// Crear un Optional con un valor
Optional<String> opt = Optional.of("4SoftwareDevelopers");

// Crear un Optional que podría ser nulo
String valorNulo = null;
Optional<String> optNullable =
    Optional.ofNullable(valorNulo);

// Si existe el valor, úsalo. Si no, usa un valor por defecto.
String resultado = optNullable.orElse("Valor por defecto");
```

Un patrón muy útil es encadenar acciones. Por ejemplo, si tienes un método que devuelve una lista que podría ser nula:


```
// Si getList() devuelve null, crea una nueva lista vacía automáticamente
List<Integer> safeList = getList().orElseGet(() -> new
ArrayList<>());
```

No es recomendable utilizar `Optional<T>` en parámetros ya que puede generar varios problemas de diseño y puede hacer tu código más difícil de entender. Sin embargo, cuando se trata de valores de retorno `Optional<T>` es una mucho mejor opción

4. Interfaces: Métodos Default y Static

Antes de Java 8, las interfaces eran contratos rígidos: sólo podían tener métodos abstractos. Si añadías un método nuevo a una interfaz, rompías todas las clases que la implementan.

Java 8 rompió esa regla permitiendo métodos con cuerpo (implementación) dentro de las interfaces:

- **Métodos `default`**: Permiten añadir nueva funcionalidad a interfaces existentes sin romper la compatibilidad.
- **Métodos `static`**: Permiten tener métodos utilitarios dentro de la interfaz.

```
public interface Ejemplo {
    // Método estático: Se llama desde la Interfaz
    static int sumar(int a, int b) {
        return a + b;
    }

    // Método default: Se hereda en la implementación
    default void imprimir(String str) {
        System.out.println("Imprimiendo: " + str);
    }
}
```

Uso:

```
// Uso del estático
int suma = Ejemplo.sumar(5, 5);

// Uso del default (requiere instancia)
Ejemplo impl = new EjemploImpl(); // Clase que implementa la
interfaz
impl.imprimir("Hola Mundo");
```

5. Interfaces Funcionales

Para que las Lambdas funcionen, Java introdujo el concepto de "Interfaz Funcional": una interfaz que tiene **exactamente un método abstracto**.

Podemos marcarlas con la anotación `@FunctionalInterface`. No es obligatoria, pero es una buena práctica porque avisa al compilador que verifique que solo exista un método abstracto.

```
@FunctionalInterface
public interface Operable {
    int ejecutar(int a, int b);
    // Si intentas añadir otro método abstracto aquí, dará
    error de compilación.
}
```

6. La Nueva API de Fecha y Hora (`java.time`)

Si alguna vez sufriste intentando formatear una fecha con `SimpleDateFormat` (que no era seguro para hilos) o sumando días con `Calendar`, entenderás por qué este cambio fue vital.

Java 8 introdujo el paquete `java.time`, inspirado en la famosa librería Joda-Time. Las nuevas clases son inmutables, seguras para hilos (thread-safe) y mucho más lógicas.

- **LocalDate**: Solo fecha (Año-Mes-Día). Ideal para cumpleaños o fechas de vencimiento.

- **LocalTime**: Solo hora.
- **LocalDateTime**: Fecha y hora combinadas (sin zona horaria).
- **ZonedDateTime**: Fecha y hora con zona horaria completa.

Ejemplo Práctico:

```
// Obtener fecha actual
LocalDate hoy = LocalDate.now();

// Crear una fecha específica (El fin del soporte de Java 8,
// por ejemplo)
LocalDate fechaEvento = LocalDate.of(2030, Month.DECEMBER,
31);

// Manipulación sencilla (Sumar/Restar)
// Antes requerías un Calendar complejo, ahora es natural:
LocalDate semanaSiguiente = hoy.plusWeeks(1);

// Parseo y Formateo (Thread-safe)
DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
String texto = hoy.format(formatter);
LocalDate fechaParseada = LocalDate.parse("25/12/2025",
formatter);
```

7. Programación Asíncrona con CompletableFuture

Antes de Java 8, manejar tareas asíncronas significaba lidiar con **Future** y bloquear el hilo principal esperando el resultado (**.get()**), o caer en el "infierno de los callbacks".

CompletableFuture trajo la promesas a Java, permitiéndonos encadenar tareas asíncronas de forma no bloqueante. Es la base de la programación reactiva que explotaría años después.

Ejemplo: Imagina buscar un usuario y luego obtener sus pedidos, todo en segundo plano:

```

CompletableFuture.supplyAsync(() -> obtenerUsuarioId(50)) //
Tarea 1
    .thenApply(id -> obtenerPedidos(id)) //
Tarea 2 (usa el resultado de 1)
    .thenAccept(pedidos -> System.out.println(pedidos)) //
Tarea 3 (consumidor final)
    .exceptionally(ex -> { //
Manejo de errores
        System.out.println("Algo falló: " + ex.getMessage());
        return null;
    });

```

8. Mejoras "Silenciosas" en Mapas

Java 8 también nos regaló métodos en la interfaz `Map` que eliminan muchísimo código repetitivo (boilerplate). El más amado por la comunidad es `computeIfAbsent`.

El "Viejo" Java (Verificar si existe):

```

Map<String, List<String>> mapa = new HashMap<>();

// Queremos agregar un valor a una lista dentro del mapa
List<String> lista = mapa.get("claves");
if (lista == null) {
    lista = new ArrayList<>();
    mapa.put("claves", lista);
}
lista.add("valor");

```

Java Moderno (Una línea):

```

// Si la clave no existe, crea la lista, la inserta y la devuelve.
// Si ya existe, solo la devuelve.
mapa.computeIfAbsent("claves", k -> new
ArrayList<>()).add("valor");

```

Conclusión: El Nuevo Estándar Mínimo

Java 8 no fue solo una actualización más; fue la demostración de que un lenguaje maduro podía reinventarse sin traicionar sus raíces. Representó el punto de inflexión donde dejamos de escribir código puramente imperativo para abrazar un estilo más declarativo, limpio y funcional.

Sin embargo, hay una verdad incómoda que debemos aceptar: en 2025 **Java 8 ya no es "lo nuevo". Es lo mínimo** o por lo menos debería serlo.

Hoy en día, dominar los Streams, las Lambdas y la API `java.time` no te convierte en un experto en Java Moderno; simplemente te saca de la categoría de "Desarrollador Legacy". Estas herramientas son ahora el vocabulario básico que necesitas para entender lo que vendrá en los siguientes capítulos.

Si Java 8 se encargó de modernizar **cómo escribimos el código**, la siguiente versión llegó para cambiar **cómo estructuramos y distribuimos nuestras aplicaciones**. Prepárate, porque en el próximo capítulo veremos cómo el lenguaje enfrentó su problema de obesidad con la llegada de la Modularidad.

05. Java 9: La Arquitectura del Futuro

Si Java 8 cambió cómo escribimos el código dentro de los métodos, Java 9 llegó para cambiar cómo **estructuramos** nuestras aplicaciones completas.

Durante años, Java sufrió de un problema de "obesidad". El JDK era un bloque monolítico gigante; incluso para ejecutar una aplicación pequeña de "Hola Mundo", tenías que cargar librerías pesadas como CORBA o Swing que probablemente jamás usarías. El **Proyecto Jigsaw** llegó para solucionar esto, dividiendo el JDK en piezas de lego (módulos) y permitiéndonos hacer lo mismo con nuestro código.

Pero Java 9 no fue solo arquitectura. También nos regaló herramientas para prototipar rápido y métodos que eliminan aún más código basura.

1. El Sistema de Módulos (Project Jigsaw)

Esta es la característica estrella. La modularidad introduce un nivel superior de encapsulamiento por encima de los paquetes. Ahora puedes ser explícito sobre qué partes de tu código son visibles para el mundo y cuáles son estrictamente internas.

Para modularizar un proyecto, agregamos un archivo `module-info.java` en la raíz del código fuente.

Ejemplo de un Módulo de Servicios:

```
module com.example.services {  
    // Exportamos este paquete para que otros lo usen  
    exports com.example.services.empresa;  
  
    // Mantenemos 'com.example.services.internal' oculto  
    (privado)  
    // No hace falta declararlo como oculto, es el default.  
}
```

Ejemplo de un Módulo Consumidor:

```
module com.example.controller {  
    // Declaramos que necesitamos el módulo de servicios  
    requires com.example.services;  
}
```

Gracias a esto, podemos crear imágenes de ejecución personalizadas ([jlink](#)) que contienen *solo* lo necesario para correr nuestra app, reduciendo drásticamente el tamaño del despliegue en la nube.

2. Colecciones Inmutables (Factory Methods)

Antes de Java 9, crear una lista pequeña e inmutable era doloroso. Tenías que usar `Arrays.asList()` (que permitía modificar elementos pero no el tamaño) o instanciar un `ArrayList` y llenarlo línea por línea.

Java 9 introdujo métodos estáticos `.of()` en las interfaces de colecciones. Esto es azúcar sintáctico puro que usamos a diario.

El "Viejo" Java:

```
List<String> lista = new ArrayList<>();  
lista.add("Java");  
lista.add("Spring");  
lista.add("Quarkus");  
lista = Collections.unmodifiableList(lista); // Para hacerla  
immutable
```

Java Moderno (Java 9+):

```
List<String> lista = List.of("Java", "Spring", "Quarkus");  
Set<Integer> numeros = Set.of(1, 2, 3);  
Map<String, Integer> mapa = Map.of("Uno", 1, "Dos", 2);
```

Nota: Estas colecciones son **inmutables**. Si intentas agregar o quitar

elementos, lanzarán `UnsupportedOperationException`.

3. JShell: Java Interactivo (REPL)

Java llegó tarde a la fiesta de los REPL (Read-Eval-Print Loop), herramientas que lenguajes como Python o Node.js tenían hace años. Pero con JShell, finalmente podemos probar código sin la ceremonia de: *Crear Archivo -> Crear Clase -> Crear main -> Compilar -> Ejecutar*.

Es perfecto para probar una expresión regular rápida o una API nueva.

Cómo usarlo: Abre tu terminal y escribe `jshell`.

```
jshell> Stream.of("Jordy", "Dev", "Java").filter(s ->
s.startsWith("J")).toList()
$1 ==> [Jordy, Java]
```

Puedes definir variables, métodos y clases al vuelo. Es una herramienta educativa y de prototipado brutal.

4. Mejoras en Stream API

Java 9 pulió la API de Streams agregando métodos que faltaban para manejar flujos condicionales de forma más natural:

- **`takeWhile(Predicate)`**: Toma elementos mientras se cumpla la condición y se detiene (como un cortocircuito).
- **`dropWhile(Predicate)`**: Descarta elementos mientras se cumpla la condición y luego toma todo el resto.
- **`ofNullable(T)`**: Crea un Stream de un solo elemento o vacío si es nulo (evita comprobaciones de nulos manuales).

Ejemplo `takeWhile`:

```
Stream.of(2, 4, 6, 8, 9, 10, 12) .takeWhile(n -> n % 2 == 0)
    .forEach(System.out::print); // Salida: 2468 // Se detiene en
    el 9 porque deja de cumplir la condición, // aunque 10 y 12
    sean pares.
```


5. Métodos Privados en Interfaces

Java 8 nos permitió tener métodos **default** y **static** en interfaces. Pero, ¿qué pasaba si dos métodos **default** compartían lógica? Teníamos que duplicar código porque no podíamos crear un método privado auxiliar.

Java 9 soluciona esto permitiendo métodos **private** en las interfaces.

```
public interface Validador {  
  
    default void validarEmail(String email) {  
        log("Validando email..."); // Reutilización interna  
        // lógica...  
    }  
  
    default void validarTelefono(String telefono) {  
        log("Validando telefono..."); // Reutilización  
        // lógica...  
    }  
  
    // Este método es solo para uso interno de la interfaz  
    private void log(String msg) {  
        System.out.println("LOG: " + msg);  
    }  
}
```

Conclusión

Java 9 fue el paso necesario para limpiar la casa. Con la modularidad, el lenguaje se volvió escalable para dispositivos pequeños y grandes nubes. Con JShell y los Factory Methods (**List.of**), se volvió más amigable para el desarrollador.

Pero el ritmo no paró aquí. En el siguiente capítulo, veremos cómo Java 10 introdujo una de las palabras clave más controversiales y amadas del lenguaje moderno: **var**.

06, Java 10: La Revolución de la Inferencia de Tipos

Java 10 fue una prueba de fuego. Fue el primer lanzamiento bajo el nuevo modelo de actualizaciones rápidas (solo seis meses después de Java 9), demostrando que el equipo de Java podía entregar valor constante sin esperas de años.

Aunque fue una versión corta, nos dejó una herramienta que cambió la estética del código Java para siempre: la inferencia de tipos en variables locales.

1. **var**: Inferencia de Tipos en Variables Locales

Probablemente la característica más amada (y al principio, temida) de esta versión. Java siempre fue criticado por ser redundante: "*¿Por qué tengo que escribir **String** dos veces si ya se ve que es un texto?*".

Con la palabra reservada **var**, el compilador infiere el tipo de la variable basándose en el valor que le asignas a la derecha.

El "Viejo" Java (Un tanto redundante):

```
String nombre = "4SoftwareDevelopers";  
ByteArrayOutputStream baos = new ByteArrayOutputStream();  
Map<String, List<Integer>> mapa = new HashMap<>();
```

Java Moderno (Conciso con **var**):

```
var nombre = "4SoftwareDevelopers"; // Infiere String  
var baos = new ByteArrayOutputStream(); // Infiere el tipo exacto  
var mapa = new HashMap<String, List<Integer>>(); // Infiere Map
```

No te recomiendo el uso de `var` en el retorno de métodos porque dificultar la lectura del código, en su lugar usalo solo en definición de variables.

✗ `var data = getData();`

✓ `var data = new ArrayList<Data>();`

Importante: Java sigue siendo fuertemente tipado No confundas `var` con el `var` de JavaScript o la tipificación dinámica de Python. Una vez que el compilador decide que la variable es un `ArrayList`, **no puedes** asignarle después un `LinkedList` o un `String`. El tipo no cambia, solo se oculta visualmente.

Lo que NO puedes hacer con `var`: El compilador necesita información suficiente para inferir el tipo, por lo que hay reglas estrictas:

- ✗ No puedes usarlo sin inicializar la variable (`var x;`).
- ✗ No puedes inicializarlo con `null` (`var x = null;`).
- ✗ No puedes usarlo en parámetros de métodos o atributos de clase (es solo para variables **locales**).

```
// Esto generará error de compilación:
var vacio;                // ¿Qué tipo es?
var nulo = null;          // ¿Es String, Integer, Object?
public void metodo(var x) // No permitido en parámetros
```

2. Colecciones Realmente Inmutables (`copyOf`)

En Java 9 vimos `List.of()`, pero ¿qué pasa si ya tienes una lista y quieres crear una copia inmutable de ella?

Java 10 introdujo métodos estáticos `copyOf` en `List`, `Set` y `Map`.

```
List<String> original = new ArrayList<>();
original.add("Java");
original.add("10");
```

```
// Crea una copia defensiva e inmutable
List<String> copiaSegura = List.copyOf(original);

// Si intentas modificar copiaSegura, lanzará
UnsupportedOperationException
// Si modificas 'original', 'copiaSegura' NO se ve afectada.
```

También se agregaron colectores para Streams que retornan colecciones inmutables directamente:

- `Collectors.toUnmodifiableList()`
- `Collectors.toUnmodifiableSet()`
- `Collectors.toUnmodifiableMap()`

3. `Optional.orElseThrow()` sin argumentos

Una pequeña mejora de la calidad de vida. Antes, si querías lanzar una excepción cuando un `Optional` estaba vacío, usabas `.get()`, pero la documentación decía que no era ideal.

Java 10 introdujo `.orElseThrow()` sin parámetros, que es la forma preferida y semánticamente correcta de obtener el valor o fallar.

```
// Preferido desde Java 10
var valor = opt.orElseThrow();

// Equivalente al antiguo (pero ahora semánticamente más
claro)
var valor = opt.get();
```

4. Mejoras de Rendimiento (G1GC)

Aunque este libro se centra en el código, vale la pena saber que Java 10 mejoró el recolector de basura G1 (Garbage Collector). Antes, el proceso de "Full GC" (limpieza completa cuando la memoria está llena) se hacía

en un solo hilo, congelando la aplicación más tiempo. Desde Java 10, esto se hace en paralelo, mejorando los tiempos de respuesta en aplicaciones críticas.

Conclusión

Java 10 fue corto pero contundente. La introducción de `var` permitió que el código Java empezara a leerse de manera más fluida, reduciendo el ruido visual de los tipos repetitivos.

Con la sintaxis limpia (Java 10) y la arquitectura modular (Java 9) listas, el escenario estaba preparado para la llegada de una de las versiones más importantes de la década: el primer gran LTS moderno. En el siguiente capítulo, exploraremos **Java 11**.

07. Java 11: El Primer LTS Moderno

Si Java 8 fue la revolución, Java 11 fue la consolidación. Al ser una versión **LTS (Long Term Support)**, se convirtió en el destino seguro para todas las empresas que querían salir de Java 8 pero no se atrevían a saltar a las versiones intermedias de ciclo corto.

Java 11 llegó con un mensaje claro: el lenguaje no solo es más moderno en sintaxis, sino que ahora tiene sus propias herramientas estándar para la web, sin depender tanto de librerías externas.

1. El Nuevo Cliente HTTP (Standard HTTP Client)

Antes de Java 11, hacer una petición HTTP nativa en Java era una tortura. Tenías que usar la arcaica `URLConnection` (que era verbosa y difícil de usar) o depender de librerías externas como Apache HttpClient o OkHttp.

Java 11 introdujo un **Cliente HTTP nativo, moderno y no bloqueante** (`java.net.http`). Soporta HTTP/2 y WebSockets de fábrica.

Ejemplo de una petición GET Sincrónica:

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()

    .uri(URI.create("https://api.github.com/users/4SoftwareDevelo
pers"))
    .GET()
    .build();

HttpResponse<String> response = client.send(request,
    HttpResponse.BodyHandlers.ofString());

System.out.println(response.body());
```

Este cliente también permite peticiones asíncronas muy elegantes usando `CompletableFuture`:

```
client.sendAsync(request,
    HttpResponse.BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenAccept(System.out::println);
```

2. Adiós a la "Ceremonia": Ejecución de Archivos Fuente

Java 11 simplificó drásticamente la curva de aprendizaje para los principiantes y el scripting rápido. Antes, para correr un programa simple, tenías que compilar (`javac`) y luego ejecutar (`java`).

Ahora, puedes ejecutar un archivo `.java` directamente. La JVM lo compila en memoria y lo ejecuta en un solo paso.

En la terminal:

```
# Antes de Java 11
$ javac Main.java
$ java Main

# Desde Java 11
$ java Main.java
```

Esto hace que Java sea viable para escribir scripts rápidos de automatización, compitiendo en terreno con Bash o Python para tareas sencillas.

3. Métodos Vitamina para la clase `String`

Java 11 añadió varios métodos de "calidad de vida" a la clase `String`. Parecen cambios pequeños, pero eliminan la necesidad de importar librerías de utilidades como *Apache Commons Lang*.

- **isBlank()**: Verifica si una cadena está vacía o solo tiene espacios en blanco. (Más inteligente que **isEmpty()**).
- **lines()**: Convierte un String multilínea en un **Stream<String>**.
- **repeat(n)**: Repite la cadena n veces.
- **strip()**: Como **trim()**, pero compatible con Unicode (elimina espacios en blanco "raros" que **trim** no detecta).

```
var texto = "  Hola Java 11  ";

System.out.println(texto.repeat(3)); // "  Hola Java 11
Hola Java 11  Hola Java 11  "
System.out.println(texto.isBlank()); // false
System.out.println(texto.strip()); // "Hola Java 11" (Mejor
que trim)

String multilinea = "Java\nPython\nNode";
multilinea.lines().forEach(System.out::println); // Procesa
línea por línea
```

4. **var** en Parámetros de Lambda

En Java 10 recibimos **var** para variables locales. En Java 11, se permitió su uso dentro de los parámetros de una expresión Lambda.

"¿Para qué querría escribir más si (**x**, **y**) ya funciona?", te preguntarás. La razón es para poder **agregar anotaciones** a los parámetros de la lambda, algo que no se puede hacer sin declarar el tipo.

```
// Permite agregar anotaciones como @NonNull
BiFunction<String, String, String> unir = (@NonNull var s1,
@Nullable var s2) -> s1 + s2;
```

Conclusión

Java 11 marcó el estándar de la industria durante años. Limpió el JDK eliminando módulos obsoletos (como CORBA y Java EE), nos dio un cliente HTTP decente y facilitó la ejecución de scripts.

Si tu proyecto corre en Java 11, estás en un terreno sólido. Pero como dice el título de este libro, el cambio es la única constante. A partir del siguiente capítulo, entraremos en el ciclo rápido de innovación que nos preparó para el siguiente gran salto. Es hora de ver cómo Java empezó a simplificar su estructura con los **Switch Expressions**.

08. Java 12: Switch Inteligente y Colectores Potentes

Java 12 es el ejemplo perfecto de una "Feature Release" intermedia: no revolucionó la plataforma entera, pero sembró las semillas de características que muchos usan a diario hoy en día. Su foco principal estuvo en el rendimiento (con la introducción experimental del recolector de basura **Shenandoah**) y en empezar a limpiar una de las estructuras más viejas de Java: el `switch`.

Pero no solo eso; también nos regaló una herramienta para Streams que muchos desarrolladores funcionales estaban pidiendo a gritos.

1. El Nuevo Switch (Switch Expressions - Preview)

El `switch` clásico de Java (heredado de C++) era propenso a errores. Si olvidabas un `break`, el código seguía ejecutándose hacia abajo (*fall-through*), y además, era muy verboso.

Java 12 introdujo las **Switch Expressions** como una característica de vista previa (*Preview Feature*). Esto nos permitió dos cosas fundamentales:

1. Usar la sintaxis de flecha `->` (lambda-style) que evita el *fall-through*.
2. Usar el `switch` como una expresión que **retorna un valor**.

El "Viejo" Switch:

```
int nivel = 0;
switch(cargo) {
    case "CEO":
    case "CTO":
        nivel = 6;
        break; // ¡Si olvidas esto, desastre!
    case "DESARROLLADOR":
```

```
        nivel = 4;
        break;
    default:
        nivel = 1;
}
```

Java Moderno (Switch Expression):

```
int nivel = switch (cargo) {
    case "CEO", "CTO" -> 6; // Múltiples etiquetas en una
    línea
    case "DESARROLLADOR" -> 4;
    default -> 1;
};
```

Nota: En esta versión, el `switch` moderno era una "Preview". Esto significa que debías activar una bandera especial para usarlo. Sin embargo, marcó el inicio de la modernización de la sintaxis que culminaría en versiones posteriores.

2. `Collectors.teeing()` (joya oculta)

A veces, cuando procesas un Stream, quieres obtener dos resultados diferentes al mismo tiempo (por ejemplo, la suma y el promedio, o el mínimo y el máximo). Antes de Java 12, tenías que recorrer la lista dos veces o crear acumuladores complejos.

`Collectors.teeing()` permite pasar dos colectores distintos y una función que combina sus resultados al final. Es como una tubería en forma de "T".

Ejemplo: Calcular el promedio de salarios (Suma / Cantidad) en un solo paso.

```

List<Double> salarios = List.of(1000.0, 2000.0, 3000.0,
4000.0);

Double promedio = salarios.stream().collect(
    Collectors.teeing(
        Collectors.summingDouble(i -> i), // Colector 1: Suma
        Collectors.counting(), // Colector 2: Cuenta
        (suma, cantidad) -> suma / cantidad // Combinador
    )
);

System.out.println(promedio); // 2500.0

```

3. Nuevos Métodos para String

Java 12 siguió mimando a la clase **String** con métodos que facilitan la manipulación de texto y la indentación.

- **indent(int n)**: Ajusta la sangría de cada línea del string. Útil para generar código o texto formateado.
- **transform(Function f)**: Permite aplicar una función a la cadena y retornar el resultado. Es útil para encadenar operaciones sin romper la fluidez del código.

```

String texto = "Java\nPython";

// Indentación
System.out.println(texto.indent(4));
/* Salida:
    Java
    Python
*/

// Transformación (chaining)
String resultado = "hola"
    .transform(s -> s + " mundo")
    .transform(String::toUpperCase);

```

```
System.out.println(resultado); // "HOLA MUNDO"
```

4. Formato de Números Compactos

Si alguna vez tuviste que programar lógica para mostrar "1K" en lugar de "1000" o "1M" en lugar de "1.000.000", Java 12 lo solucionó nativamente.

```
NumberFormat fmt = NumberFormat.getCompactNumberInstance(  
    Locale.US, NumberFormat.Style.SHORT  
);  
  
System.out.println(fmt.format(1000));    // "1K"  
System.out.println(fmt.format(1000000)); // "1M"
```

Conclusión

Java 12 nos enseñó que el lenguaje podía ser más inteligente. Aunque el **switch** moderno tardaría un par de versiones más en estandarizarse por completo (con la llegada de la palabra clave **yield** que veremos más adelante), aquí dimos el primer paso para dejar de escribir **break** en cada línea.

Además, con herramientas como **teeing()**, Java demostró que seguía apostando fuerte por la programación funcional y el procesamiento eficiente de datos. Pero la evolución no para: en el siguiente capítulo, veremos cómo Java intentó solucionar el problema de las cadenas de texto multilínea con los **Text Blocks**.

09. Java 13: Bloques de Texto y Cimientos del Futuro

Java 13 siguió puliendo las características experimentales de las versiones anteriores. Si Java 12 nos dio un `switch` más limpio, Java 13 llegó para solucionar una de las quejas más antiguas de los desarrolladores: el infierno de las cadenas de texto multilínea.

Además, bajo el capó, esta versión empezó a cambiar las tuberías internas del JDK para preparar el terreno hacia la concurrencia masiva que disfrutamos hoy.

1. Bloques de Texto (Text Blocks - Preview)

¿Alguna vez has tenido que pegar un JSON, HTML o SQL dentro de tu código Java? Antes de Java 13, esto significaba un festival de signos `+`, comillas `"` y caracteres de escape `\n` que hacían el código ilegible.

A menos claro que estuvieras usando `StringBuilder` o `StringBuffer`

Los **Text Blocks** (introducidos como Preview) nos permiten escribir cadenas multilínea de forma natural usando tres comillas dobles `"""`.

El "Viejo" Java:

```
String html = "<html>\n" +  
              "    <body>\n" +  
              "        <p>Hola Mundo</p>\n" +  
              "    </body>\n" +  
              "</html>";
```

Java Moderno (Limpio y legible):

```
String html = """  
    <html>  
    <body>
```

```
        <p>Hola Mundo</p>
    </body>
</html>
""";
```

El compilador es lo suficientemente inteligente para eliminar la indentación accidental del margen izquierdo, manteniendo el formato relativo de tu texto. Es un salvavidas para escribir consultas SQL o templates JSON.

2. Refinando el Switch: La llegada de **yield**

En Java 12, el nuevo **switch** usaba **break** para devolver valores, lo cual era confuso porque **break** se usaba históricamente para salir del flujo.

Java 13 corrigió esto introduciendo la palabra reservada **yield**. Ahora, cuando usas un bloque de código tradicional dentro de un **switch** que retorna valor, usas **yield** para "entregar" el resultado.

```
var resultado = switch (dia) {
    case LUNES, VIERNES -> 6; // Sintaxis de flecha (directa)
    case DOMINGO -> {
        System.out.println("Día de descanso");
        yield 8; // Bloque complejo: usamos yield para
retornar
    }
    default -> 0;
};
```

3. Reimplementación de la API de Sockets (JEP 353)

Hasta Java 13, la implementación interna de los Sockets (**java.net.Socket** y **ServerSocket**) era código muy antiguo, mezcla de Java y C, difícil de mantener y bloqueante.

En esta versión, se reemplazó toda esa "fontanería" por una implementación nueva y moderna (NIO). **¿Por qué te importa esto?**

Porque esta reescritura fue el **requisito técnico** para que años más tarde pudieran existir los **Virtual Threads** (Java 21). Sin este cambio invisible en Java 13, los hilos virtuales no podrían aparcarse y reanudarse eficientemente al hacer operaciones de red.

4. ZGC: Devolviendo memoria al sistema

El recolector de basura ZGC (introducido en Java 11 como experimental) recibió una mejora importante: ahora es capaz de **devolver memoria no utilizada al sistema operativo**.

Antes, ZGC tendía a acaparar la RAM y no soltarla aunque la aplicación estuviera inactiva. Este cambio hizo a Java mucho más amigable en entornos de contenedores y nube donde se paga por la memoria usada.

Conclusión

Java 13 puede parecer una actualización menor, pero fue estratégica. Nos dio la sintaxis definitiva para manejar textos largos (JSON/SQL) sin volvernos locos y preparó el núcleo del lenguaje para la revolución de concurrencia que vendría después.

Con los bloques de texto y el switch casi listos, el lenguaje estaba a punto de recibir una de las características más esperadas para reducir el código repetitivo en las clases de datos. En el siguiente capítulo, hablaremos de los **Records**.

10. Java 14: El Fin del "Boilerplate"

Si Java 8 inició la modernización, Java 14 llegó para declarar la guerra al código innecesario (el famoso "Boilerplate"). Esta versión trajo características diseñadas específicamente para que escribas menos y leas mejor.

Aquí vimos nacer oficialmente el Switch moderno y recibimos el primer vistazo de los Records, la característica que jubilaría a miles de líneas de código getter/setter.

1. Switch Expressions (Aquí se hizo oficial)

Después de dos versiones en "estado de prueba" (Java 12 y 13), en Java 14 las Expresiones Switch se convirtieron en una característica estándar y permanente del lenguaje.

Esto significa que ya puedes usarlas en producción sin miedo a que cambien. El nuevo Switch no solo es más limpio, sino que te obliga a cubrir todos los casos posibles (exhaustividad), lo que reduce bugs.

Recordatorio de la sintaxis final:

```
var resultado = switch (dia) {  
    case LUNES, VIERNES -> "Trabajo duro";  
    case DOMINGO -> "Descanso";  
    default -> "Día normal";  
};
```

2. Records (Preview): Datos, no burocracia

Nota: En Java 14 esto entró como Preview, pero fue tan revolucionario que merece el protagonismo.

Durante 25 años, si querías crear una clase simple para guardar datos (como un `Usuario` o `Producto`), tenías que escribir: campos privados, constructores, getters, setters, `equals`, `hashCode` y `toString`.

Los **Records** eliminan toda esa ceremonia. Son clases inmutables (sus datos no cambian una vez creados) y transparentes.

El "Viejo" Java (Más de 50 líneas):

```
public class PersonaDTO {
    private final String nombre;
    private final int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
    // Imagina aquí 40 líneas más de getters, equals,
    hashCode...
}
```

Java Moderno (1 sola línea):

```
public record Persona(String nombre, int edad) {}
```

El compilador genera todo por ti. Además, puedes validar datos en el constructor de forma compacta:

```
public record Persona(String nombre, int edad) {
    public Persona {
        if (edad < 0) throw new
        IllegalArgumentException("Edad inválida");
    }
}
```

No se recomienda el uso de records para modelos de dominio o entidades JPA, el uso de records es recomendado únicamente para DTOs u otros casos aislados que apliquen a tu contexto.

3. Pattern Matching para `instanceof` (Preview)

¿Cuántas veces has escrito este patrón redundante? "Si esto es un `String`, conviértelo a `String` y luego úsalo".

Java 14 introdujo el **Pattern Matching** para limpiar este proceso. Ahora, la comprobación de tipo y la conversión (casting) ocurren en el mismo paso.

El "Viejo" Java:

```
if (obj instanceof String) {  
    String s = (String) obj; // Casting manual y redundante  
    System.out.println(s.toUpperCase());  
}
```

Java Moderno:

```
if (obj instanceof String s) { // Crea la variable 's'  
    automáticamente  
    System.out.println(s.toUpperCase());  
}
```

Esto no sólo ahorra una línea; elimina la posibilidad de que hagas el casting al tipo equivocado por error.

4. `NullPointerException` (NPE) Útiles

Este es un cambio pequeño pero que te ahorrará tiempo de depuración. Históricamente, cuando recibías un `NullPointerException`, Java solo te decía: "Ocurrió un error en la línea 20". Pero si la línea 20 tenía varias operaciones encadenadas (`a.getB().getC()`), no sabías cuál era nulo.

Desde Java 14, la JVM te dice exactamente **qué variable era nula**.

Mensaje Antiguo: `Exception in thread "main"`
`java.lang.NullPointerException`

Mensaje Nuevo (Java 14+): `Cannot invoke "Ciudad.getDepartamento()" because the return value of "Persona.getCiudad()" is null`

Ahora sabes inmediatamente que el objeto `Ciudad` es el culpable, sin necesidad de usar el debugger.

Conclusión

Java 14 fue una versión centrada en la **ergonomía del desarrollador**. Con los NPEs descriptivos, depurar se volvió más fácil. Con el nuevo Switch y el `instanceof` inteligente, el código se volvió más seguro. Y con los Records, empezamos a ver un futuro donde las clases de datos son simples definiciones de una línea, no archivos gigantescos llenos de ruido.

Con estas herramientas en mano, el lenguaje estaba listo para dar el siguiente paso en seguridad y encapsulamiento. En el próximo capítulo, veremos cómo Java 15 introdujo las **Sealed Classes** para darnos control total sobre quién puede heredar nuestro código.

11 Java 15: Control de Herencia y Rendimiento Puro

Java 15 llegó con una misión clara: darnos más control. Control sobre quién puede heredar nuestro código (con las Clases Selladas) y control sobre las pausas de nuestra aplicación (con ZGC listo para producción).

Además, esta versión marcó el final del camino de prueba para los Bloques de Texto, convirtiéndolos en ciudadanos de primera clase del lenguaje.

1. Sealed Classes (Preview): El fin de la herencia descontrolada

Nota: Entró como Preview en Java 15, pero es vital entenderla desde ya.

Uno de los principios de la Programación Orientada a Objetos es la herencia, pero en Java siempre fue "todo o nada". Si hacías una clase `public`, cualquiera podía extenderla. Si la hacías `final`, nadie podía. No había un punto medio.

Las **Sealed Classes (Clases Selladas)** nos permiten definir una jerarquía restringida. Tú decides exactamente qué clases pueden ser tus hijos.

El Problema: Imagina que creas una clase `Resultado` para representar el estado de una operación. Solo quieres que sea `Exito` o `Error`. Sin clases selladas, cualquiera podría crear una tercera clase `ResultadoInesperado` y romper tu lógica.

La Solución (Java 15):

```
// Solo permitimos que Coche y Camion extiendan Vehiculo
public sealed class Vehiculo permits Coche, Camion {}

final class Coche extends Vehiculo {}
```

```
final class Camion extends Vehiculo {}
```

```
// Error de compilación: Esta clase NO está en la lista de  
// permitidos  
// class Barco extends Vehiculo {}
```

Esto permite un modelado de dominio mucho más preciso y seguro. El compilador sabe que **Vehiculo** solo tiene dos formas posibles, lo que en el futuro permitirá hacer **switch** exhaustivos sin necesidad de una cláusula **default**.

2. Bloques de Texto (¡Ya es oficial!)

Después de varias rondas de feedback en Java 13 y 14, los **Text Blocks** se convirtieron en una característica estándar en Java 15.

Ya no hay cambios de sintaxis ni riesgos de que desaparezcan. Si estás generando HTML, XML, JSON o SQL desde Java, esta es la forma definitiva de hacerlo.

```
// Ahora es 100% estándar y seguro para producción  
String json = """  
    {  
        "nombre": "Jordy",  
        "rol": "Dev",  
        "lenguaje": "Java 15"  
    }  
    """;
```

3. ZGC: Listo para Producción

El **Z Garbage Collector (ZGC)** es un recolector de basura diseñado para aplicaciones que requieren baja latencia (tiempos de respuesta rápidos) y que manejan montones de memoria gigantes (desde megabytes hasta terabytes).

En Java 15, ZGC dejó de ser experimental. Esto significa que Oracle y la

comunidad lo consideran lo suficientemente estable para correr sistemas críticos.

- **La promesa:** Pausas de recolección de basura que no superan los **10 milisegundos**, sin importar si tienes 10 MB o 10 TB de memoria.

Para activarlo:

```
java -XX:+UseZGC -jar mi-app.jar
```

4. Hidden Classes (Clases Ocultas)

Java 15 introdujo las **Hidden Classes**. Probablemente no las uses directamente en tu código del día a día porque son un poco complejas, pero son vitales para los frameworks que usas (como Spring o Hibernate).

Permiten crear clases en tiempo de ejecución que no pueden ser "vistas" por otras clases y que pueden ser descargadas de la memoria muy fácilmente. Esto ayuda a que los frameworks sean más ligeros y no llenen la memoria de "basura" que no se puede limpiar.

Conclusión

Java 15 fue una versión de madurez. Nos dio herramientas para modelar mejor nuestro negocio (Sealed Classes), limpió nuestra sintaxis (Text Blocks estándar) y nos entregó un motor de rendimiento de última generación (ZGC).

Con las piezas del tablero ya casi completas, Java se preparaba para cerrar otro ciclo. En el siguiente capítulo, veremos una versión llena de herramientas prácticas antes de llegar al gran hito del siguiente LTS. Es hora de hablar de **Java 16**.

12. Java 16: Estabilizaciones interesantes

Java 16 tuvo un rol crucial: fue la última versión antes del gran lanzamiento de largo soporte (Java 17). Su misión fue pulir, finalizar y estandarizar las características revolucionarias que habíamos estado probando en versiones anteriores.

Aquí fue donde los Records y el `instanceof` inteligente dejaron de ser experimentos para convertirse en herramientas de producción. Además, Java nos regaló una forma nativa de crear instaladores (`.exe`, `.dmg`) y dio los primeros pasos hacia el cálculo matemático de ultra-rendimiento.

1. Records y Pattern Matching (¡Ya son oficiales!)

Después de varias rondas de pruebas en Java 14 y 15, en esta versión los **Records** y el **Pattern Matching para `instanceof`** se convirtieron en características estándar del lenguaje (JEP 394 y JEP 395).

Esto significó que a partir de Java 16, podías usarlos sin activar banderas de compilación (`--enable-preview`) y con la seguridad de que la sintaxis no cambiaría en el futuro.

- **Records:** La forma definitiva de crear clases inmutables portadoras de datos.
- **Pattern Matching:** La forma limpia de hacer casting seguro.

(Si necesitas refrescar cómo funcionan, revisa el Capítulo 10 donde los introducimos).

2. `jpackage`: Instaladores Nativos

Durante años, distribuir una aplicación Java de escritorio fue un dolor de cabeza. ¿Le pides al usuario que instale el JRE? ¿Le das un JAR ejecutable?

Java 16 introdujo **jpackage**, una herramienta de línea de comandos que toma tu aplicación Java y crea un instalador nativo para el sistema operativo donde estás:

- En Windows genera **.exe** o **.msi**.
- En macOS genera **.pkg** o **.dmg**.
- En Linux genera **.deb** o **.rpm**.

Ejemplo de uso:

```
jpackage --name MiApp \  
  --input lib \  
  --main-jar main.jar \  
  --main-class com.miempresa.Main
```

Esto empaqueta tu aplicación **con su propia JVM reducida**, por lo que el usuario final no necesita tener Java instalado. ¡Es hacer doble clic y listo!

3. Vector API (Incubator)

Java empezó a “coquetear” con el hardware moderno. La **Vector API** permite realizar cálculos vectoriales (SIMD - Single Instruction, Multiple Data) que aprovechan las instrucciones especiales de las CPUs modernas para procesar múltiples datos en un solo ciclo de reloj.

Esto es vital para Inteligencia Artificial, criptografía y procesamiento de imágenes.

Java Clásico (Escalar - Un dato a la vez):

```
for (int i = 0; i < a.length; i++) {  
    c[i] = a[i] * b[i];  
}
```

Vector API (Paralelismo a nivel de CPU):

```
var vectorA = IntVector.fromArray(IntVector.SPECIES_128, a, 0);
var vectorB = IntVector.fromArray(IntVector.SPECIES_128, b, 0);
var vectorC = vectorA.mul(vectorB); // Multiplica todo de un golpe
vectorC.intoArray(c, 0);
```

Nota: En Java 16 esto entró como "Incubator" (fase experimental temprana), lo que significa que la API podía cambiar (y de hecho cambió) en versiones futuras.

4. Invocación de Métodos Default vía Reflexión

Antes de Java 16, si tenías un **Proxy Dinámico** (muy usado en frameworks como Spring o Hibernate) y querías invocar un método **default** de una interfaz, tenías que hacer malabares con hacks internos.

Java 16 añadió soporte oficial para esto en la clase **InvocationHandler**, facilitando la vida de quienes crean librerías.

```
Object proxy = Proxy.newProxyInstance(loader, interfaces,
    (p, method, args) -> {
        if (method.isDefault()) {
            // Nueva forma oficial de invocar el default
            return InvocationHandler.invokeDefault(p, method, args);
        }
        return ...;
    });
```

5. Momentos del día (B pattern)

Un pequeño detalle estético para **DateTimeFormatter**. Ahora puedes usar el patrón "B" para formatear la hora usando términos coloquiales

como "de la mañana", "de la tarde" o "de la noche", en lugar de solo AM/PM.

```
LocalTime hora = LocalTime.of(15, 0); // 3:00 PM
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("h B");
System.out.println(hora.format(fmt));
// Salida: "3 in the afternoon" (o su equivalente en español
// si configuras el Locale)
```

Conclusión

Java 16 cumplió su promesa: dejó la casa limpia y ordenada. Con los Records y el Pattern Matching ya estandarizados, y con herramientas nuevas como `jpackage`, el ecosistema estaba listo.

Todo este trabajo sentó las bases para lo que vendría a continuación: la versión que definiría el desarrollo en Java durante los siguientes años. Prepárate, porque en el próximo capítulo llega el gigante: **Java 17 LTS**.

13. Java 17: El Nuevo Estándar de la Industria

Llegamos a un hito crucial. Java 17 es una versión **LTS (Long Term Support)**, lo que significa que empresas como Oracle, Amazon y Microsoft ofrecen soporte extendido para ella (hasta 8 años en algunos casos).

Si Java 11 fue la primera parada de modernización, Java 17 es el destino definitivo para la gran mayoría de proyectos actuales. Es la versión mínima requerida para frameworks modernos como **Spring Boot 3**, y trae consigo una limpieza profunda del lenguaje y características que finalmente se graduaron.

1. Sealed Classes (Oficialmente Estándar)

Las Clases Selladas, que vimos nacer como *Preview* en Java 15, llegaron a su forma final en Java 17 (JEP 409). Ya son una parte permanente del lenguaje.

Esto es vital para el diseño de dominio. Ahora puedes restringir la herencia de manera declarativa, permitiendo que el compilador (y tus compañeros de equipo) sepan exactamente quién puede extender una clase.

Sintaxis Final:

```
public sealed interface Figura permits Circulo, Rectangulo,
Cuadrado {}

final class Circulo implements Figura {}
final class Rectangulo implements Figura {}
non-sealed class Cuadrado implements Figura {} //
'non-sealed' abre la herencia de nuevo
```

2. Encapsulamiento Fuerte de los Internos del JDK (JEP 403)

Este es el cambio "invisible" más importante y el que más dolores de cabeza causa al migrar.

Durante años, muchas librerías (y desarrolladores atrevidos) usaban clases internas del JDK (`sun.misc.*`) para hacer magia con `Unsafe` o acceder a memoria de forma privada. Esto era peligroso y dificultaba la evolución de Java.

En Java 17, se cerró el grifo. **El encapsulamiento ahora es estricto**. Si tu código intenta acceder a clases internas privadas del JDK mediante reflexión (`setAccessible(true)`), fallará con una excepción `InaccessibleObjectException`.

¿Qué hacer si tu app falla al migrar? Si usas una librería vieja que necesita acceso, tendrás que usar la bandera `--add-opens` al ejecutar, aunque lo ideal es actualizar la librería.

```
java --add-opens java.base/java.lang=ALL-UNNAMED -jar
mi-app.jar
```

3. Generadores de Números Pseudo-Aleatorios (JEP 356)

La vieja clase `java.util.Random` tenía problemas de diseño y rendimiento en entornos multihilo. Java 17 introdujo una nueva interfaz `RandomGenerator` que unifica todos los generadores y añade algoritmos modernos (como LXM) que son más rápidos y tienen mejor calidad estadística.

Java Moderno:

```
// Obtener un generador específico (L32X64MixRandom)
RandomGenerator generador =
RandomGenerator.of("L32X64MixRandom");

int numero = generador.nextInt(1, 100);
System.out.println(numero);
```

Esto facilita cambiar el algoritmo de generación sin cambiar todo tu código.

4. HexFormat (Utilidad para Hexadecimales)

Antes de Java 17, convertir bytes a una cadena hexadecimal (y viceversa) requería librerías externas o escribir bucles manuales. Ahora tenemos una clase utilitaria nativa para esto.

```
byte[] bytes = {10, 20, 30};

// De bytes a String Hexadecimal
HexFormat format = HexFormat.of();
String hex = format.formatHex(bytes);
System.out.println(hex); // "0a141e"

// De String Hexadecimal a bytes
byte[] parsed = format.parseHex("0a141e");
```

5. Pattern Matching para switch (Preview)

Java 17 introdujo como *Preview* la capacidad de usar patrones dentro del `switch`. Esto nos permite hacer comprobaciones de tipo directamente en los casos, eliminando la necesidad de cadenas `if-else` con `instanceof`.

(Nota: Como esto seguía en Preview en Java 17, lo veremos a fondo en capítulos posteriores donde se estandariza, pero es importante saber que aquí comenzó la magia).

6. Limpieza de la Casa: Adiós Applets

Java 17 marcó el final oficial de una era: la API de Applets fue marcada para eliminación (Deprecated for removal). Aunque los navegadores ya habían dejado de soportarlos hace años, esta versión confirma que Java ya no es una tecnología para correr en el navegador del cliente (frontend), sino una potencia del servidor (backend).

Conclusión

Java 17 es una versión **sólida**. Combina las mejoras de sintaxis de las versiones 12-16 (Records, Switch Expressions, Text Blocks) con la estabilidad

de un LTS y un rendimiento muy superior.

Para muchas empresas, llegar a Java 17 es la meta de modernización. Pero para nosotros, es solo la mitad del camino. A partir de aquí, Java empezó a preparar su siguiente gran revolución: los hilos virtuales. En el próximo capítulo, veremos una versión que, aunque no es LTS, trajo algo que todos esperábamos: un servidor web simple integrado. Vamos a **Java 18**.

14. Java 18: Adiós a los Problemas de Caracteres

Java 18 llegó con una misión de "higiene". Se encargó de estandarizar comportamientos que antes dependían del sistema operativo (causando errores difíciles de reproducir) y nos regaló herramientas para prototipar web y documentar mejor.

No es una versión que cambie radicalmente cómo escribes lógica de negocio, pero sí mejora significativamente tu experiencia como desarrollador y la portabilidad de tu código.

1. UTF-8 por Defecto (JEP 400)

Este es el cambio más importante y silencioso.

Antes de Java 18, la codificación de caracteres predeterminada (`Charset.defaultCharset()`) dependía del entorno donde corrías la JVM.

- En Linux y macOS solía ser **UTF-8**.
- En Windows, solía ser **Windows-1252** (o similar, dependiendo de la región).

Esto provocaba que un código que leía archivos con tildes (á, ñ, ü) funcionara perfecto en tu MacBook pero fallara al desplegarlo en un servidor Windows o viceversa.

Desde Java 18, **UTF-8 es el estándar por defecto en todas partes**, sin importar el sistema operativo .

¿Qué significa esto? Ya no necesitas especificar `StandardCharsets.UTF_8` cada vez que lees o escribes un archivo para evitar que los caracteres se rompan. Java ahora habla el idioma universal de la web de forma nativa.

2. Simple Web Server (JEP 408)

¿Alguna vez has necesitado levantar un servidor web rápido para probar un archivo HTML o compartir un fichero en tu red local, y tuviste que instalar Python (`python -m http.server`) o Node.js (`http-server`) porque Java no tenía nada fácil?

Java 18 introdujo un **Servidor Web Simple** nativo. No está diseñado para producción (no reemplaza a Tomcat, Jetty o Netty), sino para pruebas, educación y prototipos.

Cómo usarlo (Línea de comandos): Simplemente abre tu terminal y ejecuta:

```
jwebserver
```

Esto iniciará un servidor en `http://127.0.0.1:8080` sirviendo los archivos del directorio actual.

Cómo usarlo (Programáticamente): También puedes invocarlo desde código Java para pruebas unitarias o herramientas internas:

```
var server = SimpleFileServer.createFileServer(  
    new InetSocketAddress(8080),  
    Path.of("/mi/ruta/web"),  
    SimpleFileServer.OutputLevel.VERBOSE  
);  
server.start();
```

3. Javadoc @snippet (JEP 413)

Si alguna vez has escrito documentación en Java (Javadocs), sabes que incluir ejemplos de código era horrible. Tenías que usar `<pre>{@code ...}</pre>` y si querías resaltar algo, era imposible. Además, el código en los comentarios no se validaba, por lo que a menudo quedaba obsoleto y con errores.

Java 18 introdujo la etiqueta **@snippet**.

Antes (Propenso a errores):

```
/**
 * Para usarlo:
 * <pre>{@code
 * System.out.println("Hola");
 * }</pre>
 */
```

Ahora (Limpio y potente):

```
/**
 * Para usarlo:
 * {@snippet :
 * System.out.println("Hola"); // @highlight
 substring="Hola"
 * }
 */
```

Conclusión

Java 18 fue una versión de "calidad de vida". Eliminó la inconsistencia de los caracteres que nos persiguió durante 25 años y nos dio herramientas nativas para no depender de Python o Node.js en tareas sencillas.

Con la base limpia y el entorno estandarizado, el escenario estaba listo para la siguiente gran innovación en la sintaxis concurrente. En el próximo capítulo, veremos cómo Java 19 nos trajo el primer vistazo real a la revolución de los Hilos Virtuales (Virtual Threads).

15. Java 19: El Prólogo de la Revolución

Java 19 pasará a la historia no tanto por lo que finalizó, sino por lo que **comenzó**. Fue la versión que nos permitió probar, por primera vez en nuestras propias máquinas, la tecnología que cambiaría para siempre el desarrollo de aplicaciones de alto rendimiento: los Hilos Virtuales.

Además, siguió profundizando en la expresividad del lenguaje, permitiéndonos "desarmar" objetos complejos de una forma tan elegante que el código antiguo parece prehistórico.

1. Virtual Threads (Preview): El fin de la espera

Nota: Entró como Preview aquí. Se haría oficial en Java 21, pero el impacto comenzó en la 19.

Durante décadas, un "Hilo de Java" (**Thread**) equivalía 1:1 a un "Hilo del Sistema Operativo". Esto significaba que los hilos eran recursos caros y pesados. Si intentabas crear 100,000 hilos, tu servidor colapsaba por falta de memoria.

Java 19 introdujo los **Virtual Threads (Hilos Virtuales)**. Son hilos ligeros gestionados por la JVM, no por el sistema operativo. Son tan baratos que puedes crear **millones** de ellos.

El Cambio de Paradigma: Ya no necesitas complejos pools de hilos ni programación reactiva difícil de depurar para lograr alta concurrencia. Puedes escribir código bloqueante tradicional (el fácil de leer), y Java se encarga de que sea ultra-eficiente bajo el capó.

// En Java 19 (Preview), podías lanzar un millón de hilos así de fácil:

```
try (var executor =
    Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i -> {
        executor.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1)); // No
            // bloquea el OS
            return i;
        });
    });
}
```

```

    });
  });
}
// ¡Esto se ejecuta en segundos sin consumir toda tu RAM!

```

2. Record Patterns (Preview): Deconstrucción de Datos

Ya vimos que los **Records** son geniales. Pero, ¿qué pasa si queremos acceder a sus datos internos rápidamente?

Antes, tenías que verificar el tipo, hacer casting y luego llamar a los getters. Java 19 introdujo los **Patrones de Record**, que nos permiten "deconstruir" el objeto en el mismo momento en que verificamos su tipo.

Antes de Java 19:

```

if (obj instanceof Persona p) {
    // Aún tengo que llamar a los métodos manualmente
    String nombre = p.nombre();
    int edad = p.edad();
    System.out.println(nombre + " tiene " + edad);
}

```

Java 19 (Deconstrucción directa):

```

// Persona(String nombre, int edad)
if (obj instanceof Persona(String nombre, int edad)) {
    // ¡Las variables 'nombre' y 'edad' ya existen y
    tienen valor aquí!
    System.out.println(nombre + " tiene " + edad);
}

```

Esto funciona incluso con records anidados, permitiéndote extraer datos profundos en una sola línea.

3. Structured Concurrency (Incubator)

Junto con los hilos virtuales, Java 19 trajo (en fase incubadora) una nueva forma de pensar en tareas paralelas: la **Concurrencia Estructurada**.

La idea es simple: si una tarea se divide en subtareas (hilos), esas subtareas deberían tratarse como una sola unidad de trabajo. Si la tarea padre se cancela, todas las hijas deberían cancelarse automáticamente.

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure())
{
    Future<String> usuario = scope.fork(() ->
    buscarUsuario());
    Future<Integer> ordenes = scope.fork(() ->
    buscarOrdenes());

    scope.join();           // Espera a ambos
    scope.throwIfFailed();  // Si uno falla, propaga el error

    // Aquí ya tenemos ambos resultados seguros
    return new Respuesta(usuario.resultNow(),
    ordenes.resultNow());
}
```

Esto elimina el riesgo de "hilos huérfanos" que siguen consumiendo recursos cuando ya no son necesarios.

4. Pattern Matching para Switch (Third Preview)

El **switch** siguió evolucionando. En Java 19 se refinó el uso de patrones. Se introdujo la cláusula **when** (guardas) para añadir condiciones lógicas dentro de un caso del switch.

```
switch (obj) {  
    case String s when s.length() > 5 ->  
System.out.println("Texto largo: " + s);  
    case String s -> System.out.println("Texto corto: " + s);  
    default -> System.out.println("Otra cosa");  
}
```

Conclusión

Java 19 fue el "tráiler" de la película que todos queríamos ver. Nos dejó jugar con los Hilos Virtuales y nos mostró cómo la Concurrencia Estructurada podía simplificar el código paralelo complejo.

Aunque muchas de estas características aún requerían activar banderas (`--enable-preview`), la comunidad empezó a migrar mentalmente hacia este nuevo modelo. El camino estaba pavimentado para las siguientes versiones, donde estas herramientas se pulirían hasta la perfección. Pero antes, Java 20 llegaría para dar el último toque de refinamiento.

16. Java 20: La “calma” antes de la tormenta

A primera vista, Java 20 parece una versión tranquila. No introdujo grandes características nuevas desde cero, sino que se dedicó a incubar y refinar las que ya existían (Pattern Matching, Vector API, Foreign Function API).

¿Por qué? Porque el objetivo era dejar todo impecable para el lanzamiento del siguiente LTS (Java 21). Sin embargo, hubo movimientos importantes bajo la superficie, especialmente una nueva forma de compartir datos entre hilos que jubiló patrones de diseño de hace 20 años.

1. Scoped Values (Incubator): El reemplazo de ThreadLocal

Esta característica es crucial si vas a usar Hilos Virtuales.

Durante años, usamos `ThreadLocal` para pasar datos (como el ID de usuario o una transacción) a través de la pila de llamadas sin pasarlos como argumentos en cada método. Pero `ThreadLocal` tiene un problema grave: es mutable, pesado en memoria y difícil de limpiar.

Si vas a tener millones de Hilos Virtuales, `ThreadLocal` es inviable (demasiada memoria). Java 20 introdujo los **Scoped Values** (Valores de Alcance) como una alternativa moderna: son inmutables, ligeros y se limpian automáticamente cuando termina la tarea.

```
final static ScopedValue<String> USUARIO =
    ScopedValue.newInstance();
// En algún lugar del servidor:
ScopedValue.where(USUARIO, "Jordy")
    .run(() -> {
        // Dentro de este bloque (y sus sub-métodos),
        USUARIO vale "Jordy"
```

```
System.out.println("Usuario actual: " + USUARIO.get());
    procesarPeticion();
});
// Aquí USUARIO ya no existe. No hay fugas de memoria.
```

(Nota: Al ser Incubator, requiere `--enable-preview --add-modules jdk.incubator.concurrent`)

2. Record Patterns: Inferencia de Genéricos (2nd Preview)

Los patrones de Records (que vimos en Java 19) recibieron una mejora de inteligencia. Ahora el compilador puede inferir los tipos genéricos dentro del patrón, ahorrándote aún más código.

Java 19 (Verbolidad genérica):

```
// Tenías que ser explícito si el Record usaba genéricos
if (obj instanceof Caja<String>(String contenido)) { ... }
```

Java 20 (Inferencia):

```
// Java infiere que 'var' es del tipo correcto dentro del
genérico
if (obj instanceof Caja(var contenido)) {
    System.out.println(contenido);
}
```

3. Un consejo práctico: Deja de usar `java.net.URL`

Este es un cambio pequeño pero que afecta a mucho código "legacy". Los constructores de la clase `URL` fueron marcados como **Deprecados**.

¿La razón? La implementación de `URL` hace operaciones bloqueantes (como resolución DNS) en lugares inesperados, lo cual es terrible para el

rendimiento moderno. La recomendación oficial desde Java 20 es usar **URI** para construir la dirección y luego convertirla.

El "Viejo" Java (Evítalo):

```
URL url = new URL("https://www.4softwaredevelopers.com"); //  
¡Deprecado!
```

Java Moderno:

```
URL url =  
URI.create("https://www.4softwaredevelopers.com").toURL();
```

4. Pattern Matching for Switch (4th Preview)

El culebrón del **switch** continuó. En Java 20 se lanzó la **cuarta** vista previa. Esto demuestra el cuidado obsesivo que tiene el equipo de Java: no lo hicieron estándar hasta estar 100% seguros de que manejaba todas las excepciones y casos borde correctamente. En esta versión, se mejoró el manejo de errores cuando un enum o clase sellada cambia y el switch deja de ser exhaustivo (**MatchException**), preparando el terreno para su estandarización final.

Conclusión

Java 20 fue el último paso de preparación. Limpió el camino para la concurrencia ligera (Scoped Values), pulió la sintaxis de patrones y nos advirtió sobre el uso de clases viejas como **URL**.

Todo estaba listo. Las herramientas estaban afiladas, las APIs probadas y la comunidad expectante. En el próximo capítulo, llegamos al momento cumbre de la década para el ecosistema Java: el lanzamiento de **Java 21 LTS** y la llegada oficial de los Virtual Threads.

17. Java 21: La Era de Oro (LTS)

Si Java 8 fue el renacimiento, Java 21 es la consagración. Esta versión LTS representa la culminación de años de trabajo en el Proyecto Loom, el Proyecto Amber y el Proyecto Panama.

Es la versión que elimina las excusas para usar otros lenguajes en tareas de alta concurrencia. Con Java 21, el lenguaje se vuelve ligero, expresivo y capaz de manejar escalas masivas de tráfico sin despeinarse.

1. Virtual Threads (Oficial): Escalabilidad Infinita

La característica que mató a la programación reactiva compleja.

Históricamente, los hilos de Java (`java.lang.Thread`) eran wrappers 1:1 de los hilos del sistema operativo (OS Threads). Eran pesados (2MB de RAM aprox.) y costosos de crear. Si tu servidor recibía 1 millón de peticiones simultáneas, necesitabas 1 millón de hilos, lo cual colapsaba cualquier máquina.

Los **Virtual Threads** (JEP 444) rompen esa relación 1:1. Ahora, la JVM mapea millones de hilos virtuales sobre un puñado de hilos del sistema operativo.

¿Cuándo usarlos? Son perfectos para tareas I/O Bound (bases de datos, llamadas a APIs, leer archivos). Cuando un hilo virtual se bloquea esperando una respuesta, la JVM lo "aparca" y usa el hilo real para otra tarea. ¡Es literalmente concurrencia gratuita!

Ejemplo: 10,000 Tareas en 1 segundo Antes, esto hubiera requerido un Pool de Hilos limitado. Ahora:

```
// Creamos un Executor que genera un Hilo Virtual nuevo por
// cada tarea
try (var executor =
    Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i -> {
        executor.submit(() -> {
            // Simulamos una operación de red de 1 segundo
            Thread.sleep(Duration.ofSeconds(1));
            return i;
        });
    });
}
// Tiempo total de ejecución: ~1 segundo (porque corren en
// paralelo real)
```

El cambio es tan simple como cambiar `newFixedThreadPool` por `newVirtualThreadPerTaskExecutor`. No tienes que aprender una API nueva compleja.

2. Sequenced Collections (Colecciones Secuenciadas)

Java tenía un desorden histórico en sus colecciones. Algunas tenían método `get(0)`, otras `getFirst()`, otras no tenían forma fácil de ir al revés.

Java 21 introduce tres interfaces nuevas (`SequencedCollection`, `SequencedSet`, `SequencedMap`) que unifican el acceso al primer y último elemento.

El "Viejo" Java (Inconsistente):

```
var primerElemento = list.get(0);
var ultimoElemento = list.get(list.size() - 1); // ¡Qué feo!
```

Java 21 (Estándar y Limpio):

```
var primero = list.getFirst();
var ultimo = list.getLast();
```

```
var listaInvertida = list.reversed(); // Vista invertida de la lista
```

Esto aplica para `ArrayList`, `LinkedList`, `LinkedHashSet`, `TreeSet` y sus equivalentes en Mapas (`putFirst`, `putLast`, `pollFirstEntry`).

3. Pattern Matching Finalizado (Switch y Records)

Después de años en "Preview", en Java 21 el Pattern Matching para Switch y los Record Patterns son oficiales.

Esto nos permite escribir código polimórfico extremadamente limpio, eliminando cadenas de `if-else` y casteos manuales.

El "Viejo" Java:

```
if (obj instanceof String s) { ... }  
else if (obj instanceof Integer i) { ... }
```

Java 21 (Switch Pattern Matching):

```
String resultado = switch (obj) {  
    case String s when s.length() > 5 -> "Texto largo: " +  
s.toUpperCase();  
    case String s -> "Texto corto: " + s.toLowerCase();  
    case Integer i -> "Es un número: " + (i * i);  
    case Posicion(int x, int y) -> "Es un Record en: " + x +  
", " + y;  
    case null -> "Vino nulo"; // ¡Ahora podemos manejar null en el switch!  
    default -> "Desconocido";  
};
```

Observa dos cosas claves:

1. **Deconstrucción de Records:** `case Posicion(int x, int y)` extrae los valores directamente.

2. **Guardas (when):** Permiten añadir lógica extra al caso `(s.length() > 5)`.

4. Simplificación del "Main" (Preview)

Una guiño a los principiantes. Java 21 introdujo (como Preview) la posibilidad de escribir clases sin tanta ceremonia. Aunque para producción seguiremos usando clases completas, para aprender es genial.

Antes:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hola");  
    }  
}
```

Java 21 (Preview):

```
void main() {  
    System.out.println("Hola mundo simplificado");  
}
```

Esto elimina la barrera de entrada de `public static void main(String[] args)` para quienes recién empiezan.

Conclusión

Java 21 no es solo una versión más; es el nuevo estándar. La introducción de los **Virtual Threads** eliminó la mayor desventaja de Java frente a lenguajes como Go o Node.js (la concurrencia ligera), mientras que las **Sequenced Collections** y el **Pattern Matching** terminaron de pulir la experiencia de desarrollo diaria.

Si hoy empiezas un proyecto nuevo, **Java 21 es la casilla de salida obligatoria**. Pero la innovación no se detiene en los LTS. En el siguiente capítulo, veremos cómo Java 22 empezó a preparar el terreno para la interoperabilidad nativa sin dolor.

18. Java 22: Código Limpio y Sin Fronteras

Después de la resaca de la fiesta de Java 21 LTS, el equipo de Java no descansó. Java 22 llegó para solucionar esos pequeños detalles molestos que todos hemos ignorado durante años (como declarar variables que no usamos) y para derribar definitivamente la barrera entre Java y el código nativo (C/C++).

En esta versión, el lenguaje se vuelve más intencional: si algo no te importa, ahora puedes decirlo explícitamente.

1. Variables y Patrones sin Nombre (_) (JEP 456)

Esta es la característica de "calidad de vida" definitiva.

¿Cuántas veces has tenido que declarar una variable en un **try-catch** o en un bucle que jamás usaste? Los IDEs te avisaban con una advertencia amarilla, pero el lenguaje te obligaba a ponerle nombre.

Java 22 finalizó el uso del **guion bajo** (_) para indicar variables que **existen, pero no nos importan**.

Caso 1: Excepciones ignoradas

```
// Antes: Teníamos que inventar nombres como "ignored" o "e"
try {
    int numero = Integer.parseInt(texto);
} catch (NumberFormatException e) { // 'e' no se usa, pero ensucia
    System.out.println("No es un número");
}

// Java 22: Intencionalidad clara
try {
    int numero = Integer.parseInt(texto);
} catch (NumberFormatException _) { // ¡El guion bajo lo dice
    System.out.println("No es un número");
}
```

Caso 2: Pattern Matching en Switch Si solo quieres saber el tipo, pero no necesitas el valor:

```
switch (obj) {  
    case Integer _ -> "Es un entero, no me importa cuál";  
    case String _ -> "Es un texto, no me importa qué dice";  
    default -> "Otra cosa";  
}
```

Caso 3: Bucles

```
// Ejecutar algo 10 veces sin usar el índice  
for (int _ : lista) {  
    System.out.println("Procesando elemento...");  
}
```

2. Foreign Function & Memory API (FFM) - ¡Oficializada! (JEP 454)

Alternativa a JNI.

Durante décadas, si querías llamar a una librería escrita en C o C++ desde Java, tenías que usar **JNI (Java Native Interface)**. Aunque con sus más y sus menos lo cierto es que requiere escribir código "pegamento" en C.

Java 22 finalizó la **API FFM**. Ahora puedes llamar a funciones nativas y gestionar memoria fuera del Heap (**off-heap memory**) usando **puro código Java**.

Ejemplo: Llamar a la función **strlen de C estándar** Sin escribir ni una línea de C:

```
// 1. Buscador de librerías nativas  
Linker linker = Linker.nativeLinker();  
SymbolLookup stdlib = linker.defaultLookup();  
  
// 2. Buscamos la función "strlen" en la librería estándar del OS  
MemorySegment strlenAddress = stdlib.find("strlen").orElseThrow();
```

```
// 3. Definimos la firma (Recibe un Puntero, Devuelve un Long)
FunctionDescriptor descriptor = FunctionDescriptor.of(JAVA_LONG,
ADDRESS);

// 4. Creamos el manejador
MethodHandle strlen = linker.downcallHandle(strlenAddress,
descriptor);

// 5. Usamos un "Arena" para gestionar la memoria segura
try (Arena arena = Arena.ofConfined()) {
    // Convertimos String Java a C-String (puntero)
    MemorySegment cString = arena.allocateFrom("Hola Java 22");

    // Invocamos la función nativa
    long longitud = (long) strlen.invoke(cString);
    System.out.println("Longitud calculada por C: " + longitud);
// 12
}
```

Esto abre las puertas para usar librerías de IA (TensorFlow, PyTorch) o drivers de hardware directamente desde Java con alto rendimiento y seguridad.

Es importante mencionar que esto no busca reemplazar JNI, de hecho a día de hoy mucha gente usa JNI, se siguen creando paquetes que la usan.

3. Statements before super(...) (Preview) (JEP 447)

Una regla histórica de Java era: "En un constructor, la llamada a `super()` debe ser la primera línea". Esto nos impedía validar argumentos antes de pasárselos al padre.

Java 22 relajó esta regla (como Preview). Ahora puedes ejecutar lógica antes de llamar al padre, siempre que no toques `this`.

Antes (Con métodos estáticos auxiliares):

```
public class Hijo extends Padre {
    public Hijo(int valor) {
```



```

        super(validar(valor)); // Tenías que crear un método
estático 'validar'
    }
}

```

Java 22 (Lógico y directo):

```

public class Hijo extends Padre {
    public Hijo(int valor) {
        if (valor < 0) throw new
IllegalArgumentException("¡Negativo no!");
        System.out.println("Validación exitosa");
        super(valor); // Ahora sí llamamos al padre
    }
}

```

4. Stream Gatherers (Preview) (JEP 461)

La API de Streams es genial (**filter**, **map**, **reduce**), pero siempre le faltó flexibilidad para operaciones intermedias complejas, como agrupar elementos en ventanas o transformar el flujo de formas personalizadas.

Los Gatherers son para las operaciones intermedias lo que los **Collectors** son para las finales.

Ejemplo: Agrupar en ventanas (chunks) Imagina que tienes una lista [1, 2, 3, 4, 5, 6] y quieres procesarla en grupos de 3: [1, 2, 3] y [4, 5, 6]. Antes esto era una pesadilla en Streams.

```

// Requiere importar java.util.stream.Gatherers
List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);

var grupos = numeros.stream()
    .gather(Gatherers.windowFixed(3)) // ¡Magia! Ventanas de
3
    .toList();

System.out.println(grupos); // [[1, 2, 3], [4, 5, 6]]

```

Conclusión

Java 22 demostró que el lenguaje sigue enfocado en la **ergonomía**. La variable sin nombre `_` limpia el ruido visual que arrastramos por años, y la API FFM nos conecta con el mundo nativo sin los dolores de cabeza del pasado.

Con estas herramientas, Java se volvió más expresivo y potente. Pero el ciclo de innovación de 6 meses no se detiene. En el siguiente capítulo, veremos qué nos trajo **Java 23** y cómo el lenguaje sigue adoptando características modernas como los tipos primitivos en patrones.

19. Java 23: Documentación Humana y Simplicidad

Java 23 llegó con un regalo para todos los que odiamos escribir HTML dentro del código Java. Sí, leíste bien. Durante casi 30 años, escribir Javadoc significaba llenar tu código de etiquetas `<p>`, `<ul>`, `<li>` y `<pre>` que hacían el código fuente difícil de leer.

Esta versión modernizó eso, además de simplificar drásticamente cómo importamos librerías y cómo manejamos los tipos primitivos en los patrones.

1. Javadoc en Markdown (JEP 467)

El fin de HTML en los comentarios.

Hasta Java 23, la documentación se veía así:

El "Viejo" Javadoc (HTML):

```
/**
 * <p>Calcula el &acute;rea.</p>
 * <ul>
 * <li>Opci&ocirc;n A</li>
 * <li>Opci&ocirc;n B</li>
 * </ul>
 * <pre>{@code
 * return x * y;
 * }</pre>
 */
public int calcular() { ... }
```

Era propenso a errores. Java 23 introdujo oficialmente el soporte para **Markdown** en los comentarios de documentación, usando la nueva sintaxis de tres barras `///`.

Java 23 (Markdown limpio):

```
/// Calcula el área.  
///  
/// - Opción A  
/// - Opción B  
///  
/// ```java  
/// return x * y;  
/// ```  
public int calcular() { ... }
```

El compilador de Javadoc ahora entiende Markdown estándar. Tu código fuente se ve limpio, y la documentación generada se ve igual de profesional.

2. Importación de Módulos (Preview) (JEP 476)

Adiós a las listas infinitas de imports.

¿Te ha pasado que abres un archivo Java y las primeras 30 líneas son solo `import java.util.List;`, `import java.util.Map;`, `import java.util.stream.Stream;`, etc.?

Java 23 introdujo la capacidad de **importar un módulo completo** de una sola vez.

```
import module java.base;  
  
// ¡Ya puedes usar TODO lo que está en el módulo base!  
// List, Map, Stream, File, Path, LocalDate... sin imports  
// extra.  
  
public class Ejemplo {  
    List<String> lista = new ArrayList<>();  
    Path ruta = Path.of("archivo.txt");  
}
```

Esto no afecta el rendimiento (el compilador es inteligente y solo carga lo que usas), pero mantiene tus archivos increíblemente limpios, especialmente al prototipar.

3. Tipos Primitivos en Patrones (Preview) (JEP 455)

El Pattern Matching y los Switch modernos eran geniales, pero tenían una limitación molesta: no funcionaban bien con tipos primitivos (`int`, `long`, `double`). Tenías que usar sus wrappers (`Integer`, `Long`).

Java 23 eliminó esta barrera. Ahora puedes usar primitivos directamente en `instanceof` y `switch`.

Switch con primitivos:

```
long peso = 500L;

switch (peso) {
    case 1L -> System.out.println("Un gramo");
    case long l when l > 1000 -> System.out.println("Pesado: " + l);
    default -> System.out.println("Normal");
}
```

Instanceof con primitivos (Seguro de tipos): Esto parece extraño, pero es útil cuando trabajas con tipos genéricos o conversiones. Java verifica si el valor *cabe* en el tipo destino sin perder información.

```
int valor = 123;
// Verifica si 'valor' se puede tratar como byte (rango -128 a 127)
if (valor instanceof byte b) {
    System.out.println("Cabe en un byte: " + b);
} else {
    System.out.println("Es demasiado grande para un byte");
}
// Salida: Cabe en un byte: 123
```

4. Simplificación para Principiantes: **IO** (Third Preview)

Como parte de la iniciativa para hacer Java más amigable a los nuevos estudiantes (junto con el `void main()` que vimos antes), Java 23 añadió la clase `java.io.IO`.

Permite leer y escribir en la consola sin la ceremonia de `System.out.println` o `Scanner`.

```
import module java.base;

void main() {
    IO.println("¿Cómo te llamas?");
    String nombre = IO.readLine("");
    IO.println("Hola, " + nombre);
}
```

Conclusión

Java 23 nos recordó que el código no es solo para las máquinas; es para los humanos. La capacidad de escribir documentación en Markdown y de limpiar los imports hace que trabajar con el lenguaje sea mucho más agradable visualmente.

Pero la rueda no deja de girar. Con el año 2025 acercándose en el horizonte, Java se preparaba para otra actualización de rendimiento. En el próximo capítulo, veremos **Java 24** y cómo llevó la API de Streams a un nuevo nivel de paralelismo.

20. Java 24: La Pieza Faltante de los Streams

Lanzada en marzo de 2025, Java 24 llegó con una misión específica: hacer que la programación funcional en Java fuera, por fin, completa.

Durante años, amamos los Streams (`.map`, `.filter`), pero siempre nos topábamos con un muro cuando queríamos hacer operaciones "intermedias" complejas, como agrupar elementos en ventanas deslizantes o transformar el flujo de maneras no estándar. Java 24 solucionó esto estandarizando los **Gatherers**. Además, introdujo cambios profundos en cómo Java lee sus propios archivos, lo que disparó el rendimiento de arranque de frameworks como Spring.

1. Stream Gatherers (Oficial)

La mayor actualización a Streams desde 2014.

Hasta ahora, la API de Stream tenía un punto débil: las operaciones intermedias eran fijas (`filter`, `map`, `limit`...). Si querías algo personalizado, tenías que escribir un `Collector` complejo al final, o peor, volver a un bucle `for`.

Los **Gatherers** (JEP 461, estandarizado aquí) te permiten crear tus propias operaciones intermedias.

El Problema (Antes de Java 24): Imagina que tienes una lista de eventos y quieres procesarlos en parejas (Ventana de 2): `[A, B, C, D] -> [A, B], [B, C], [C, D]`. Con Streams clásicos, esto era casi imposible de hacer limpio.

La Solución (Java 24):

```
import java.util.stream.Gatherers;

List<String> eventos = List.of("Login", "Click", "Compra",
    "Logout");

var parejas = eventos.stream()
    .gather(Gatherers.windowSliding(2)) // ¡Nativo!
    .toList();

// Salida: [[Login, Click], [Click, Compra], [Compra,
Logout]]
System.out.println(parejas);
```

No solo sirve para ventanas. Puedes crear Gatherers personalizados para, por ejemplo, detener el stream si encuentras un error pero emitir lo que llevas procesado hasta ese momento (algo que `takeWhile` no hace exactamente igual).

2. Class-File API (Preview Avanzada)

El cambio invisible que acelera tu framework favorito.

Java depende mucho de leer archivos `.class` (bytecode). Frameworks como Spring, Hibernate o Mockito usaban librerías de terceros (como ASM) para esto. El problema es que cada vez que salía una versión nueva de Java, estas librerías se rompían y tenías que esperar a que las actualizaran.

Java 24 introdujo una API nativa robusta (`java.lang.classfile`) para leer, escribir y transformar archivos de clase.

¿Por qué te importa? Aunque no uses esto directamente, significa que:

1. Tus dependencias se rompen menos al actualizar Java.
2. El arranque de aplicaciones complejas es más rápido porque la JVM hace el trabajo sucio nativamente sin capas extra.

3. Reducción de "Pinning" en Virtual Threads

Mejorando la concurrencia.

Uno de los pocos problemas de los **Virtual Threads** (Java 21) era el "Pinning". Si un hilo virtual entraba en un bloque **synchronized**, se quedaba "pegado" al hilo del sistema operativo, bloqueándolo y perdiendo la ventaja de rendimiento.

En Java 24, se reescribieron gran parte de los monitores de objetos (**synchronized**) para reducir drásticamente este problema.

El Resultado: Ya no tienes que correr a cambiar todos tus **synchronized** por **ReentrantLock** para usar hilos virtuales. El código antiguo (legacy) ahora corre mucho mejor sobre la nueva arquitectura de hilos ligeros de forma automática.

4. String Templates (El retorno)

Nota: Esta característica tuvo una historia turbulenta (retirada en la 23), pero volvió rediseñada.

Java 24 trajo de vuelta una forma segura de interpolar Strings, enfocada en la seguridad contra inyección (SQL, HTML). En lugar de simplemente concatenar variables, el enfoque de Java es usar procesadores de plantillas.

```
String nombre = "Jordy";
String tabla = "usuarios";

// STR es un procesador estándar
String mensaje = STR."Hola \{nombre}, bienvenido.";

// Procesadores seguros (hipotético SQL)
// Esto valida o escapa los valores automáticamente
Statement stmt = SQL."SELECT * FROM \{tabla} WHERE user =
\{nombre}";
```

Conclusión

Java 24 fue la versión de la **flexibilidad**. Con los Gatherers, eliminamos los últimos casos donde nos veíamos obligados a abandonar los Streams. Y con las mejoras en Virtual Threads, el código antiguo se volvió moderno mágicamente sin reescribirlo.

Todo esto nos dejó en la puerta de entrada del presente. Las herramientas están listas, la casa está limpia y el rendimiento es máximo. Es hora de cerrar este viaje histórico con la versión en la que vivimos hoy: **Java 25 LTS**.

21. Java 25: La Cuspide del Rendimiento (LTS)

Hemos llegado al final del viaje histórico y nos encontramos en el presente. Java 25 no es solo otra versión de soporte a largo plazo; es la versión que **derriba las barreras físicas** de la memoria.

Durante 30 años, en Java "todo fue un Objeto" (menos los primitivos). Esto tenía un costo: cada vez que creabas un objeto pequeño (como un `Punto(x, y)`), la JVM tenía que asignar memoria en el Heap, crear una cabecera, gestionar su identidad y usar punteros. Eso desperdiciaba memoria y ciclos de CPU.

Java 25 integra las primeras grandes victorias del **Project Valhalla** y **Project Lilliput**, haciendo que el lenguaje sea más eficiente a nivel de hardware que nunca.

1. Value Classes (Clases de Valor) - Early Access/Standard

El sueño del Project Valhalla hecho realidad.

Imagina que pudieras crear una clase que se comporte como un `int`. Sin identidad (no puedes usar `==` para ver si son la misma instancia, solo si valen lo mismo), sin punteros innecesarios, y que se almacene "plana" en memoria.

Java 25 introduce las **Value Classes**.

El Problema:

```
// Cada 'Punto' es un objeto completo con cabecera y puntero.  
// Un array de Puntos es un array de referencias (Lento).  
Punto[] puntos = new Punto[1000];
```

La Solución (Java 25):

```
public value class Punto(int x, int y) {
```

```
// Comportamiento de clase, rendimiento de primitivo
}  
  
// En memoria, esto se aplana. Es como tener un array de ints  
consecutivos.  
// Cero "pointer chasing" (saltos de memoria).
```

Esto dispara el rendimiento en aplicaciones matemáticas, financieras y de IA, acercando a Java a la eficiencia de memoria de C++ o Rust, pero con la seguridad de siempre.

2. Compact Object Headers (Project Lilliput)

Rendimiento invisible y gratuito.

¿Sabías que antes de Java 25, cada objeto vacío en Java ocupaba entre 96 y 128 bits solo en "encabezados" (metadata de la JVM)? Aunque tu objeto solo tuviera un **byte** de datos, el envoltorio era gigante.

Java 25 implementa **Compact Headers** (Cabeceras Compactas), reduciendo este tamaño a 64 bits (o incluso menos en ciertas configuraciones).

¿Qué significa esto para ti?

- No tienes que cambiar ni una línea de código.
- Al actualizar a Java 25, tu aplicación podría consumir automáticamente **un 10% o 20% menos de RAM**.
- En aplicaciones con millones de objetos pequeños, el ahorro es brutal.

3. Pattern Matching en Bucles **for**

El Pattern Matching ya conquistó el **switch** y el **instanceof**. En Java 25, finalmente llega a los bucles, permitiéndonos iterar y deconstruir al mismo tiempo.

Antes (Verboso):

```
for (Object obj : listaMezclada) {  
    if (obj instanceof String s) {  
        System.out.println("Texto: " + s);  
    }  
}
```

Java 25 (Elegante y limpio):

```
// Solo itera sobre los elementos que coincidan con el patrón  
for (String s : listaMezclada) {  
    System.out.println("Texto: " + s);  
}  
  
// O deconstruyendo Records directamente  
for (Punto(var x, var y) : listaDePuntos) {  
    System.out.println("Coordenadas: " + x + ", " + y);  
}
```

4. Computed Constants (Project Leyden - Preview)

Arranque instantáneo.

Uno de los grandes esfuerzos recientes es mejorar el tiempo de arranque (Startup Time) para competir en entornos Serverless (AWS Lambda). Java 25 introduce el concepto de constantes calculadas en tiempo de compilación/linkado.

La JVM ahora es capaz de guardar el estado de ciertas variables estáticas "congeladas" en el archivo ejecutable, para no tener que recalcularlas cada vez que la aplicación arranca. Esto es la base para que los frameworks como Spring inicien en milisegundos sin necesidad de compilación nativa compleja (GraalVM).

Conclusión del Viaje

Desde los días oscuros de Java 6, pasando por la revolución funcional de Java 8, la modularidad de Java 9, hasta la era moderna de los Hilos Virtuales (Java 21) y las Clases de Valor (Java 25).

Java ha demostrado algo que muy pocos lenguajes logran: **saber envejecer**. No se quedó estancado; observó, aprendió y se adaptó.

Hoy, en 2025, tienes en tus manos un lenguaje que:

1. Es **conciso** (**var**, **record**, **switch** patterns).
2. Es **ligero** (Virtual Threads, Value Classes).
3. Es **rápido** (ZGC, FFM, Lilliput).

Ya no hay excusas. El "Java Viejo" ha muerto. ¡Larga vida al Java Moderno!

22. GraalVM y AOT: La Revolución Nativa

Durante 25 años, el funcionamiento de Java fue siempre el mismo: tu código se compila a **Bytecode** (`.class`) y luego una Máquina Virtual (JVM) lo interpreta y compila a código máquina mientras la aplicación corre (**JIT - Just in Time**).

Este modelo es fantástico para aplicaciones de larga duración porque la JVM se vuelve más inteligente con el tiempo y optimiza el código en caliente. Pero tiene un precio: **el arranque lento** y el **alto consumo de memoria**.

En la era de Kubernetes y las funciones Serverless (AWS Lambda), no podemos esperar 5 segundos a que una aplicación arranque. Aquí es donde entra **AOT (Ahead-of-Time Compilation)**.

1. ¿Qué es AOT y GraalVM?

AOT significa compilar tu código Java directamente a **código máquina binario** (como si fuera C++ o Go) *antes* de ejecutarlo.

GraalVM es la herramienta que hizo esto popular con su utilidad **Native Image**. Toma tu aplicación, analiza todo lo que usa, elimina lo que no usa, y genera un ejecutable nativo (`mi-app.exe` o binario de Linux).

La Diferencia es Brutal:

Característica(JIT)	JVM Tradicional	GraalVM Native Image (AOT)
Arranque	1.5 - 10 segundos	0.05 segundos (Instantáneo)
Memoria RAM	256 MB+ (solo por arrancar)	30 MB - 50 MB
Archivo	Requiere instalar Java (JRE)	Autocontenido (No requiere Java)

Rendimiento	Mejor a largo plazo (Peak)	Mejor al inicio, estable
-------------	----------------------------	--------------------------

2. El cambio de mentalidad: "Mundo Cerrado"

La compilación nativa tiene una regla de oro: **La Asunción de Mundo Cerrado**.

En el Java tradicional, puedes cargar clases dinámicamente que ni siquiera existían cuando compilaste (Reflexión dinámica). En AOT, el compilador debe conocer **todo** el código que se va a ejecutar en el momento de la compilación.

Esto significa que:

1. La **Reflexión** requiere configuración extra (aunque frameworks como Spring Boot 3 o Quarkus ya lo hacen automático por ti).
2. No puedes descargar JARs de internet y ejecutarlos al vuelo.

3. Frameworks "Cloud Native"

La popularidad de AOT ha dado nacimiento a una nueva generación de frameworks diseñados específicamente para ser compilados nativamente:

- **Quarkus:** Se autodenomina "Supersonic Subatomic Java". Fue el pionero en hacer que AOT fuera fácil.
- **Micronaut:** Diseñado desde cero sin usar reflection runtime.
- **Spring Boot 3+:** Desde noviembre de 2022, Spring soporta oficialmente AOT y GraalVM, permitiendo que millones de aplicaciones existentes se vuelvan nativas.

4. ¿Cuándo usar AOT y cuándo JIT?

No caigas en el error de pensar que AOT siempre es mejor.

- **Usa AOT (Nativo):**
 - Si usas **Serverless** (necesitas arrancar en milisegundos).

- Si usas **Contenedores** (Docker/K8s) y quieres meter muchos en un servidor pequeño (ahorro de RAM).
- Para herramientas de línea de comandos (CLI).
- **Usa JIT (JVM Tradicional):**
 - Para aplicaciones monolíticas grandes y complejas.
 - Para sistemas de misión crítica de muy larga duración (días/meses encendidos), ya que el compilador JIT (C2) suele lograr un rendimiento pico (throughput) ligeramente superior al binario estático.

Conclusión

AOT y GraalVM han eliminado la única excusa que quedaba para no usar Java en la nube moderna: el consumo de recursos. Hoy, Java puede ser tan ligero como Go y tan rápido en arrancar como un script de Bash, manteniendo la robustez y el ecosistema que amamos.

Con esto cerramos el ciclo técnico del lenguaje. Hemos visto su historia, su sintaxis moderna y su nueva forma de ejecución. ¿Qué nos queda? Entender cómo aprender todo esto sin perderse.

23. Herramientas para Sobrevivir al Cambio

Saberse la sintaxis de memoria está muy bien, pero en el día a día, la batalla real es gestionar el entorno y mantener el código limpio.

Si vas a trabajar con Java Moderno, no puedes seguir descargando instaladores `.exe` o `.dmg` de la página de Oracle manualmente, ni puedes pretender refactorizar un proyecto gigante línea por línea a mano.

Aquí te presento las tres herramientas que todo desarrollador Java moderno debería utilizar.

1. SDKMAN!: Domina el Multiverso de Versiones

Es probable que en tu trabajo tengas que mantener un proyecto "legacy" en Java 8, otro en Java 17 y quieras probar las novedades de Java 25 en tus ratos libres. Manejar las variables de entorno (`JAVA_HOME`, `PATH`) manualmente es un infierno.

SDKMAN! (Software Development Kit Manager) es una herramienta de terminal que te permite instalar, cambiar y gestionar versiones de Java (y Maven, Gradle, Spring Boot, etc.) al instante.

Instalación (Linux/macOS/WSL):

```
curl -s "https://get.sdkman.io" | bash
```

Uso:

```
# ¿Quieres instalar la última versión de Amazon Corretto?
$ sdk install java 21.0.1-amzn

# ¿Quieres instalar GraalVM?
$ sdk install java 21.0.1-graal

# ¿Cambiar de versión para la sesión actual?
```

```
$ sdk use java 8.0.392-tem
```

Con SDKMAN!, cambiar de proyecto es cuestión de segundos.

2. OpenRewrite: Migración Automatizada

Aquí está el secreto mejor guardado para migrar de Java 8 a Java 21.

OpenRewrite no es un linter que te dice qué está mal; es una herramienta que **reescribe tu código automáticamente (usalo con cuidado)**.

Puede:

- Cambiar `new ArrayList<>()` por `List.of()`.
- Actualizar JUnit 4 a JUnit 5.
- Migrar propiedades de Spring Boot 2 a 3.
- Eliminar código muerto.

Se ejecuta como un plugin de Maven o Gradle:

```
mvn -U org.openrewrite.maven:rewrite-maven-plugin:run \
-Drewrite.recipeArtifactCoordinates=org.openrewrite.recipe:rewrite-spring:RELEASE \
-Drewrite.activeRecipes=org.openrewrite.java.spring.boot3.UpgradeSpringBoot_3_2
```

Ejecutas eso, y “*listo*”: tu código se actualiza solo. Obviamente requiere revisión humana, pero te puede ahorrar una parte importante del trabajo tedioso.

3. JBang: Scripts en Java sin dolor

Mencionamos en capítulos anteriores que Java 11 permitía ejecutar archivos `.java` directamente. **JBang** lleva esto al siguiente nivel. Te permite escribir scripts en Java que declaran sus propias dependencias en la cabecera.

¿Quieres hacer un script que use una librería JSON y haga una petición HTTP, pero no quieres crear un proyecto Maven completo con `pom.xml`?

script.java:

```
//DEPS com.fasterxml.jackson.core:jackson-databind:2.15.2
//DEPS com.squareup.okhttp3:okhttp:4.11.0

import okhttp3.*;
// ... código normal ...
```

Ejecutar:

```
jbang script.java
```

JBang descarga las librerías en una caché oculta y ejecuta el código. Es perfecto para automatización, pruebas rápidas y herramientas CLI.

Epílogo. Tu Hoja de Ruta Personalizada

Hemos llegado al final de este viaje. Hemos recorrido casi dos décadas de innovación, desde las expresiones Lambda de Java 8 hasta la optimización de memoria de Java 25.

Es normal que en este punto sientas una mezcla de emoción y vértigo. La pregunta lógica es: *¿Tengo que implementar todo esto mañana mismo?*

La respuesta corta es: **No**.

Java es un ecosistema inmenso. Intentar aplicar Hilos Virtuales, Clases Selladas, AOT y la API de Memoria Foránea al mismo tiempo en un proyecto que apenas está migrando de Java 8 es la receta perfecta para el desastre.

Para ayudarte a digerir este libro y aplicar lo aprendido de manera estratégica, he diseñado tres **Rutas de Aprendizaje**. Identifica en qué momento se encuentra tu proyecto (o tu carrera) y sigue el camino que más te convenga.

Ruta 1: La Misión de Rescate (Migración)

Para ti si: Estás atrapado en un monolito Java 8/11 y tu objetivo es actualizarte sin romper nada.

Esta ruta prioriza la estabilidad y la limpieza del código base. No te preocupes por el rendimiento extremo todavía; preocúpate por la deuda técnica.

1. **El Cimiento:** Domina el **Capítulo 04 (Java 8)**. Asegúrate de que los *Streams* y *Optional* se usen correctamente, no como bucles glorificados.
2. **La Estandarización:** Salta al **Capítulo 06 (Java 10)** y **Capítulo 14 (Java 18)**. Empieza a usar **var** para limpiar la lectura y asegura que tu proyecto use UTF-8 por defecto.

3. **El Salto:** Tu meta es **Java 21 LTS (Capítulo 17)**. No intentes ir a la 25 todavía. La 21 es el estándar de oro actual para estabilidad.
4. **La Herramienta Clave:** Usa **OpenRewrite (Capítulo 23)**. No migres a mano. Automatiza la actualización de JUnit, las propiedades de Spring y los tipos de Java.

Ruta 2: El Arquitecto de Software (Diseño)

Para ti si: Estás diseñando un nuevo sistema, una librería o definiendo el dominio de tu negocio.

Aquí el foco es la expresividad, la seguridad de tipos y evitar que el código se vuelva inmanejable con el tiempo.

1. **Datos Limpios:** Los **Records (Capítulo 10 y 16)** son tu nueva unidad básica. Úsalos para todos tus DTOs y eventos.
2. **Control de Dominio:** Implementa **Sealed Classes (Capítulo 13)**. Define jerarquías estrictas donde tú controlas quién extiende a quién.
3. **Lógica Expresiva:** Domina el **Pattern Matching (Capítulo 21 y 25)**. Elimina los `if-else` anidados y escribe lógica de negocio que se lea como un libro.
4. **Estructura:** Repasa la **Modularidad (Capítulo 05)**. Aunque no uses módulos estrictos, aplica sus principios de encapsulamiento fuerte.

Ruta 3: Rendimiento Cloud Native (Escalabilidad)

Para ti si: Trabajas con microservicios de alto tráfico, entornos Serverless (AWS Lambda) o necesitas exprimir cada ciclo de CPU.

Esta es la ruta avanzada. Aquí asumimos que ya tienes un código limpio y buscas velocidad pura.

1. **Concurrencia Masiva:** Tu Biblia es el **Capítulo 17 (Virtual Threads)**. Migra tus pools de hilos tradicionales a hilos virtuales para manejar miles de peticiones simultáneas.

2. **Eficiencia de Memoria:** Estudia el **Capítulo 25 (Java 25)**. Entiende cómo las *Value Classes* y los *Compact Headers* pueden reducir tu factura de la nube.
3. **Arranque Instantáneo:** Implementa **GraalVM y AOT (Capítulo 22)**. Haz que tus servicios arranquen en milisegundos.
4. **Interoperabilidad:** Si necesitas IA o drivers rápidos, usa la **API FFM (Capítulo 18 y 22)** para hablar con el código nativo sin dolor.

Palabras Finales

Escribí este libro porque estoy convencido de una cosa: **Java no es el lenguaje del pasado; es el lenguaje del futuro.**

Pocos ecosistemas han logrado sobrevivir 30 años y seguir siendo líderes indiscutibles. La clave ha sido su capacidad de adaptación. Java supo observar a Python y aprender concisión; supo mirar a Go y aprender concurrencia ligera; supo mirar a Rust y aprender eficiencia de memoria.

Ahora, la responsabilidad de adaptarse es tuya.

No permitas que el miedo a "romper algo" te mantenga atado a versiones de hace una década. La tecnología avanza, y tú debes avanzar con ella. Usa este libro como tu guía de consulta, experimenta, rompe cosas en desarrollo (para eso está) y, sobre todo, nunca dejes de aprender.

¿Tienes dudas o recomendaciones? Búscame en: YouTube:

/[4SoftwareDevelopers](#) *Twitter/X:* [@jordy_4sd](#)

Nos vemos pronto.

Jordy Andrés Rodríguez Arango

Versión Diciembre, 2025